

UNIVERSITATEA „SAPIENTIA” DIN CLUJ-NAPOCA
FACULTATEA DE ȘTIINȚE TEHNICE ȘI UMANISTE,
TÎRGU-MUREȘ
SPECIALIZAREA CALCULATOARE

BUS4U: Sistem modern de transport public -
UI
PROIECT DE DIPLOMĂ

Coordonator științific:

Ș.l.dr.ing. Szántó Zoltán

Absolvent:

Portik Szabolcs

2025

BUS4U: Sistem modern de transport public - UI

Extras

În zilele noastre, transportul public este rar utilizat din cauza lipsei de confort, a ineficienței sau a neîndeplinirii așteptărilor utilizatorilor. Problemele frecvente includ autobuze care nu respectă orarele sau dificultăți în achiziționarea biletelor. În multe locuri, orarele nu sunt afișate în stații și nici disponibile online. Se întâmplă adesea ca autobuzele să fie aglomerate, iar călătorii află acest lucru prea târziu, când nu au alte opțiuni de transport. Platforma online Bus4U își propune să rezolve aceste probleme.

Bus4U constă dintr-o aplicație mobilă, un site web pentru utilizatori și o interfață web de administrare. Funcțiile sale principale includ căutarea rutelor de autobuz prin specificarea punctului de plecare, a destinației și a orei, achiziționarea biletelor online în avans și vizualizarea orelor de plecare defalcate pe orașe și stații. În plus, stațiile de autobuz pot fi vizualizate pe o hartă filtrată după orașe și rute, iar pozițiile actuale ale autobuzelor pot fi urmărite în timp real. Pentru a asigura aceste funcționalități și gestionarea biletelor, o aplicație dedicată rulează pe terminalele POS instalate în autobuze. Această aplicație validează biletele și partajează informațiile live despre autobuze cu serverul central.

În plus, aplicația web oferă o interfață de administrare pentru companiile de autobuze. Aici, administratorii pot gestiona orarele, stațiile și încărcă rutele. Administratorii companiilor pot gestiona, de asemenea, informațiile angajaților și diverse date despre autobuze, cum ar fi inspecțiile tehnice periodice și asigurările. Este disponibilă și o interfață statistică, care permite conducătorilor companiilor să vizualizeze numărul de utilizatori înregistrați, biletele vândute și veniturile generate, defalcate pe lună, an sau pe întreaga perioadă.

Posibilitățile de dezvoltare viitoare includ introducerea abonamentelor și a diverselor reduceri, precum și urmărirea în timp real a gradului de ocupare a autobuzelor. Din perspectiva companiilor de autobuze, monitorizarea programului de lucru al șoferilor și implementarea contabilității automate ar putea fi, de asemenea, benefice.

Această lucrare se concentrează pe interfața utilizatorului a software-ului, incluzând cele două site-uri web și aplicația mobilă, în timp ce funcționalitatea backend este acoperită într-o altă lucrare.

Cuvinte cheie: transport în comun, cursă de autobuz, digitalizare

SAPIENTIA ERDÉLYI MAGYAR TUDOMÁNYEGYETEM
MAROSVÁSÁRHELYI KAR
SZÁMÍTÁSTECHNIKA SZAK

Bus4U: Modern tömegközlekedési rendszer -
UI
DIPLOMADOLGOZAT

Témavezető:

Dr. Szántó Zoltán

Egyetemi adjunktus

Végzős hallgató:

Portik Szabolcs

2025

Kivonat

Napjaink felgyorsult világában kevesen használják a tömegközlekedést, mert az nem elég kényelmes, nem elég gyors vagy éppen nem úgy működik, ahogy azt elvárnánk tőle. A gyakori problémák közé tartozik, hogy a buszok nem tartják a menetrendet vagy a jegyek megvásárlása nehézkes. Sok helyen a menetrend nincs kifüggesztve a megállóban és az interneten sem elérhető. Az is gyakran megesik hogy a buszok nagyon zsúfoltak és erről az utazni vágyók csak akkor vesznek tudomást, amikor már a megállóban vannak és nem áll rendelkezésre más utazási lehetőség. A Bus4U online platform ezen problémákra kínál megoldást.

A Bus4U egy mobilalkalmazásból, egy felhasználói weboldalból és egy adminisztrátori weboldalból tevődik össze. Főbb funkciói közé tartozik a buszjáratok keresése a kiindulópont, a célállomás és az időpont megadásával, a járatokra szóló előzetes online jegyvásárlás, illetve a járatok indulási időpontjainak megtekintése városokra és megállókra lebontva. Ezen kívül megtekinthetők a busz-megállók a térképen, városok és útvonalak szerint szűrve, valamint valós időben követhetők az autóbuszok pillanatnyi helyzetei is. Ezen funkciók biztosításához, illetve a jegykezeléshez, szükség van egy alkalmazásra, ami a buszokon található POS terminálon fut. Ez biztosítja a jegyek érvényesítését, illetve az autóbuszok élő információinak megosztását is a központi szerverrel.

Ezen felül a webalkalmazás biztosít egy adminisztrációs felületet is a busztársaságok számára. Itt lehetőség nyílik elsősorban a menetrendek és a megállók kezelésére, valamint az útvonalak feltöltésére. A busztársaság adminisztrátora itt tudja kezelni az alkalmazottak információit, és az autóbuszok különböző adatait is, mint az időszakos műszaki vizsga és a biztosítás. Ugyanitt található egy statisztikai felület is, ahol a busztársaságok vezetői megtekinthetik a regisztrált felhasználók számát, az eladott jegyek számát és az ezekből generált bevételt, mindezt havi, éves és mindenkori bontásban.

Továbbfejlesztési lehetőségek között szerepelnek a bérletek és különböző kedvezmények létrehozása, valamint a buszok telítettségének követése valós időben. A busztársaságok szempontjából fontos lehet még a sofőrök munkaidejének követése és az automatizált könyvelés bevezetése is.

Ez a dolgozat a szoftver felhasználói felületére fókuszál, a két weboldalra és a mobilalkalmazásra, míg a backend működése egy másik dolgozat keretein belül van ismertetve.

Kulcsszavak: tömegközlekedés, buszjárat, digitalizálás

Abstract

In today's fast-paced world, public transportation is rarely used due to its lack of comfort, inefficiency, or failure to meet users' expectations. Common issues include buses not adhering to schedules or difficulties in purchasing tickets. In many places, schedules are neither displayed at stops nor available online. It often happens that buses are overcrowded, and passengers only realize this when they are already at the stop without having alternative travel options. The Bus4U online platform aims to solve these problems.

Bus4U consists of a mobile application, a client website, and an administrator web interface. Its main features include searching for bus routes by specifying the departure point, destination, and time, purchasing tickets online in advance, and viewing departure times broken down by cities and stops. Additionally, bus stops can be viewed on a map filtered by cities and routes, and buses' current positions can be tracked in real time. To ensure these functionalities and manage tickets, a dedicated application runs on POS terminals installed on buses. This app validates tickets and shares live bus information with the central server.

Furthermore, the web application provides an administration interface for bus companies. Here, administrators can manage schedules, stops, and upload routes. Company administrators can also manage employee information and various bus-related data, such as periodic technical inspections and insurance. A statistical interface is also available, allowing company executives to view the number of registered users, sold tickets, and generated revenue in monthly, yearly, or all-time breakdowns.

Future development possibilities include introducing subscriptions and various discounts, as well as tracking bus occupancy in real time. From the perspective of bus companies, monitoring drivers' working hours and implementing automated accounting could also prove beneficial.

This thesis focuses on the software's user interface, including the two websites and the mobile application, while the backend functionality is covered in another thesis.

Keywords: public transport, bus ride, digitalization

Tartalomjegyzék

1. Bevezető	5
2. Célkitűzések	7
3. Szakirodalmi tanulmány	9
3.1. Létező megoldások	13
3.1.1. Moovit	13
3.1.2. Budapest Go	14
4. A rendszer specifikációi	16
4.1. Felhasználói követelmények	16
4.1.1. Felhasználói felület	17
4.1.2. Adminisztrátori felület	18
4.2. Rendszerkövetelmények	20
4.2.1. Funkcionális követelmények	20
4.2.2. Nem funkcionális követelmények	22
5. A rendszer architektúrája	24
5.1. Frontend	25
5.2. Backend	26
5.3. Kommunikáció	27
5.4. Felhasznált technológiák	31
5.4.1. Vue.js	31

5.4.2.	Vuetify	33
5.4.3.	Flutter	33
5.5.	Fejlesztői eszközök	34
5.6.	Kitelepítés	36
6.	A felhasználói felület tervezése	37
6.1.	Material Design	37
6.2.	Figma terv	38
7.	Gyakorlati megvalósítás	42
7.1.	Felhasználói weboldal és alkalmazás	42
7.1.1.	Főoldal	42
7.1.2.	Jegyek	45
7.1.3.	Menetrend	46
7.1.4.	Megállók és élő térkép	48
7.2.	Adminisztrátori weboldal	49
7.2.1.	Kezdőoldal	49
7.2.2.	Megállók	50
7.2.3.	Útvonalak	50
7.2.4.	Menetrend	51
7.2.5.	Buszok és alkalmazottak	52
8.	Tesztelés	54
8.1.	Cypress	55
8.2.	Megvalósított tesztek	55
8.3.	Automatizált tesztelés	58
9.	Összefoglalás	60
9.1.	További fejlesztési lehetőségek	61
	Irodalomjegyzék	62

Ábrák jegyzéke

3.1. Intelligens Közlekedési Rendszer	11
3.2. Moovit alkalmazás	14
3.3. Budapest Go alkalmazás ¹	15
4.1. Use-case diagram a végfelhasználók lehetőségeiről	19
4.2. Use-case diagram az adminisztrátorok lehetőségeiről	20
5.1. A rendszer architektúrája	24
5.2. Azonosítást tartalmazó API kérés	25
5.3. A jegyek listázásának válasza	28
5.4. A jegyek listázásának folyamata	28
5.5. A regisztrációra adott válasz	29
5.6. A regisztráció teljes folyamata	30
5.7. Feladatok a Jira-ban	35
5.8. Verziókövetés GitKraken segítségével	36
6.1. A Material Design 3 gomb stílusai	38
6.2. A felhasználói weboldal terve	40
6.3. Az admin weboldal terve	40
6.4. A mobilalkalmazás terve	41
7.1. Jegyvásárlás a weboldalon	43
7.2. Jegyvásárlás a mobilalkalmazásban	43
7.3. A jegyvásárlás folyamata	44

7.4. A felhasználó megvásárolt jegyei	46
7.5. Listázott menetrend és az útvonal térképe	47
7.6. Megállók	48
7.7. Kezdőoldal statisztikával és értesítéssel	49
7.8. Adminisztrátori megállók oldal	50
7.9. Útvonalak létrehozása, módosítása és törlése	51
7.10. Menetrend szerkesztése	52
7.11. Az alkalmazottak listája	53
7.12. Busz szerkesztése és alkalmazott létrehozása	53
8.1. A felhasználói weboldal tesztjei	56
8.2. A GitHub Actions által generált teszteredmények	59

1. fejezet

Bevezető

A tömegközlekedési rendszerek sajnos nem túl népszerűek az utazók körében, egyrészt az autók kényelme miatt, másrészt a tömegközlekedésben jelenlévő problémák miatt. Ezen problémák közé tartoznak a buszok pontatlansága, a zsúfoltság és a hiányos információk. Sok helyen a buszok nem is rendelkeznek menetrenddel, vagy ha igen, azok nem pontosak. Digitalizáció szempontjából is rengeteg hiányosság van a tömegközlekedésben, így ez az egyik legelavultabb szolgáltatások egyike.

Jelenleg már ott tartunk, hogy az emberek többsége okostelefonnal rendelkezik, különböző alkalmazásokat használ és böngészik az interneten. Ezt rengeteg vállalkozás ki is használja, hogy felgyorsítsa és egyszerűsítse a szolgáltatásait. A tömegközlekedésben is egyre több vállalkozás próbálja megoldani a problémákat, de még mindig sok a hiányosság. Ezeket belátva nem is csoda, hogy az emberek inkább az autózást választják, mint a tömegközlekedést.

Egy jól megtervezett és kivitelezett tömegközlekedési rendszer sokat segítene úgy az embereknek, mint a környezetnek. Egyre több problémát okoz ugyanis a globális felmelegedés, amelynek egyik okozója az autók károsanyag kibocsátása. A tömegközlekedés egyik legnagyobb előnye, hogy sok ember szállításával kevesebb szennyező anyag kerül a levegőbe, mint az autózás esetén. Ezen kívül a tömegközlekedés sokkal kisebb területet foglal el, mint az autózás, így a városokban is sokkal kevesebb helyet foglal el a parkolás. Egyes városokban már most is problémát okoz a parkolóhelyek hiánya és a hatalmas dugók, amelyeket az autók okoznak. Ezt mindenki tapasztalja nap mint nap, de sajnos így is kevesen választják a tömegközlekedést.

Azért választottuk ezt a témát Zsigmond Attila kollégámmal, mert mindketten megtapasztaltuk

a tömegközlekedés és a városi forgalom problémáit a mindennapi ingázás során és úgy gondoltuk, hogy egy digitalizált rendszer sokat segíthetne a hozzánk hasonló ingázóknak és az alkalmi utazóknak is. Ha az emberek interneten tudnának tájékozódni a buszokról és a forgalomról, intézhetnék itt a jegyvásárlást, és láthatnák a buszok valós idejű pozícióját, valószínűleg sokkal többen választanák a tömegközlekedést az autózás helyett.

A dolgozat célja egy olyan rendszer bemutatása, amely a digitalizáció által megoldja a tömegközlekedésben jelen lévő problémákat, ezáltal kényelmesebbé és egyszerűbbé téve a tömegközlekedést. Ez a rendszer nem csak az utasoknak nyújthat segítséget, hanem a busztársaságoknak is, mert ezt használva egyszerűbben tudják intézni online az ügyeiket és jobban át tudják tekinteni a vállalatukat.

A szóban forgó rendszer neve Bus4U, amely az angol Bus For You rövidítése és amely jelentése magyarul: Buszjárat Neked. A Bus4U egy végfelhasználóknak szánt platformból és egy adminisztrátori felületből tevődik össze. A felhasználói felület főbb funkciói közé tartozik a buszjáratok keresése, a járatokra szóló előzetes online jegyvásárlás, illetve a járatok indulási időpontjainak megtekintése városokra és megállókra lebontva. Ezen kívül megtekinthetők a buszmegállók a térképen, városok és útvonalak szerint szűrve, valamint valós időben követhetőek az autóbuszok pillanatnyi helyzetei is.

Az adminisztrációs felületen lehetőség nyílik elsősorban a menetrendek és a megállók kezelésére, valamint az útvonalak feltöltésére. A busztársaság adminisztrátora itt tudja kezelni az alkalmazottak információit, és az autóbuszok különböző adatait is, mint az időszakos műszaki vizsga és a biztosítás. Ugyanitt található egy statisztikai felület is, ahol a felhasználók számát, az eladott jegyek számát és az ezekből generált bevételt lehet megtekinteni havi, éves és mindenkori bontásban.

A rendszer két nagy részre osztható fel. Az egyik a backend, amely a háttérfolyamatokért és az üzleti logikáért felelős, míg a másik a frontend, amely a felhasználók számára látható felületet kezeli. A backend működése a kollégám diplomamunkájában kerül ismertetésre [1], míg az én dolgozatom a frontend részt mutatja be, amely magában foglalja a weboldalakat és a mobilalkalmazást.

Ebben a dolgozatban szó lesz a frontend követelményeiről, az architektúráról, a felhasználói felület tervezéséről és a megvalósításról. Be fogom mutatni a felhasznált eszközöket és technológiákat, a tervezési és implementációs folyamatot, valamint a végeredményt is. Beszélni fogok a tesztelésről és a továbbfejlesztési lehetőségekről, végül pedig a levonható következtetésekről is.

2. fejezet

Célkitűzések

Fő célunk egy olyan rendszer kialakítása, amely egyaránt hasznos a tömegközlekedést választó embereknek, illetve a busztársaságoknak. A backend specifikus célok a Bus4U - Server [1] dolgozatban vannak ismertetve, míg itt a frontend specifikus célok vannak inkább kiemelve. A teljes rendszer működése érdekében a következő célokat kell teljesíteni a felhasználók kiszolgálásának szempontjából:

- könnyen használható weboldal, illetve mobilalkalmazás elkészítése, amely gyors és egyszerű hozzáférést biztosít a tömegközlekedési információkhoz
- egységes és jól kinéző felület biztosítása a különböző platformokon
- útvonaltervezés és jegyvásárlás: a felhasználók megadhatják a kiindulópontjukat, a célállomásukat és az időpontot, majd a rendszer megmutatja a lehetséges járatokat és azok árait, valamint lehetőséget biztosít a jegyvásárlásra
- menetrendek megtekintésének lehetősége: indulási időpontok megjelenítése, megállók és járatok szerint
- buszok útvonalának és megállóinak megtekintése: az útvonalak és megállók térképen való megjelenítése, és szűrési lehetőség biztosítása városok és útvonalak szerint
- buszok valós idejű követése: folyamatosan frissülő adatok megjelenítése a buszok pillanatnyi helyzetéről a térképen

A busztársaságok szempontjából további funkciókra is szükség van, amelyek biztosítják az adminisztrátorok számára a cég menedzseléséhez szükséges eszközöket. Ehhez a következő célok elérésére van szükség:

- egy könnyen használható adminisztrációs felület létrehozása, amely csak a busztársaságok adminisztrátorai számára elérhető
- menetrendek és megállók kezelése: menetrendek és megállók hozzáadása, módosítása és törlése
- útvonalak feltöltése: útvonalak hozzáadása, módosítása, megállók hozzárendelése és törlése
- jegyárak beállítása: útvonalakhoz, illetve útszakaszokhoz
- alkalmazottak és buszok adatainak kezelése: alkalmazottak és buszok hozzáadása, módosítása és törlése
- statisztikai adatok megtekintése: regisztrált felhasználók száma, eladott jegyek száma és bevétel, többféle bontásban
- POS terminálon futó alkalmazás elkészítése és buszokra való kihelyezése, amely biztosítja a jegyvásárlást, a jegyek érvényesítését és valós időben figyeli a busz földrajzi pozícióját

3. fejezet

Szakirodalmi tanulmány

A dolgozat témájának megértéséhez először meg kell vizsgálni néhány alapfogalmat, amelyek a dolgozatban szereplő rendszerekhez kapcsolódnak. Itt bemutatásra kerülnek a klasszikus tömegközlekedési rendszerek és a modernebb Intelligens Közlekedési Rendszerek (ITS)¹, valamint a jelenleg elérhető megoldások, mint a Moovit és a Budapest Go alkalmazások, amelyek a városi közlekedés digitalizálására törekednek. Mindezek előtt azonban érdemes megvizsgálni a tömegközlekedés használatának pozitív hatásait, hogy megértsük, miért is fontos ezzel foglalkozni.

Manapság már mindenki tapasztalja a globális felmelegedés és a környezetszennyezés hatásait, amelyek egyik fő oka a közlekedési eszközök károsanyag kibocsátása. Egy 2018-as felmérés szerint a közlekedési szektor tette ki a globális szén-dioxid kibocsátás 25%-át, ennek pedig túlnyomó része a közúti közlekedésből származik [2]. A közlekedési eszközök károsanyag kibocsátásának csökkentése érdekében egyre több városban és országban támogatják a tömegközlekedés használatát, mert ezáltal csökkenthető a környezetszennyezés és a városi dugók is. Több fejlődő országban elvégzett tanulmány alátámasztja a tömegközlekedési eszközök bevezetésével csökkenő légszennyezést. Például Tajpej első metróvonalának megnyitása 5-15%-al csökkentette a levegő CO₂ tartalmát [2].

A tömegközlekedés nem csak a környezetvédelem szempontjából fontos, hanem a városi forgalom csökkentésében is fontos szerepet játszik. A tömegközlekedés használata csökkentheti a városi dugókat azáltal, hogy az emberek egy járműben utaznak sok személyautó helyett, így kevesebb lesz a járművekkel elfoglalt hely mérete. Egyelőre nem sokan választják a tömegközlekedést, a kisebb

¹ Intelligent Transport System

kényelem miatt, de megfelelő fejlesztésekkel vonzóbbá lehet tenni azt, így csökkenthető lehet a városi forgalom és a környezetszennyezés is [3].

A városi tömegközlekedés javításával kapcsolatban már a 70-es években is felmerült az igény a buszok automatizált kezelésére, amely akkor a megállóban való torlódást szeretne volna megoldani. Ekkor használták először a Busz Flotta Kezelés (BFM)² kifejezést a buszok kezelésére. Később ezt a menetrendek kezelésére is alkalmazták, amely a buszok forgalmát és a járatok menetrendjét is magában foglalta. Napjainkban további feladatokkal egészül ki a tömegközlekedés menedzsmentje, amelyek úgynevezett Intelligens Közlekedési Rendszereket (ITS) hoznak létre. Ide tartozik a balesetmegelőzés, a buszok pozícióinak követése, az energiakezelés és a fenntarthatóság is. Egyes alrendszernek saját megnevezéseik vannak, mint például a Valós Idejű Információ (RTI)³, amely az utasokat tájékoztatja valós időben a közlekedés állapotáról és az esetleges változásokról, vagy az Automatikus Jármű Helyzet (AVL)⁴ rendszer, amely a közlekedési eszközök földrajzi pozícióit követi és ezek alapján végez számításokat. A klasszikus BFM-et kibővítve az ITS-ben található elemekkel egy olyan összetett busz kezelő rendszert kapunk, amely az alap szolgáltatások mellett, lehetővé teszi a buszok helyzeteinek valós idejű követését és az utasok számára a valós idejű információk megjelenítését, és amelynek együttes megnevezése a Dynamic Bus Fleet Management [4, 5].

Az ITS rendszerek általában a következő fő komponensekből állnak [5]:

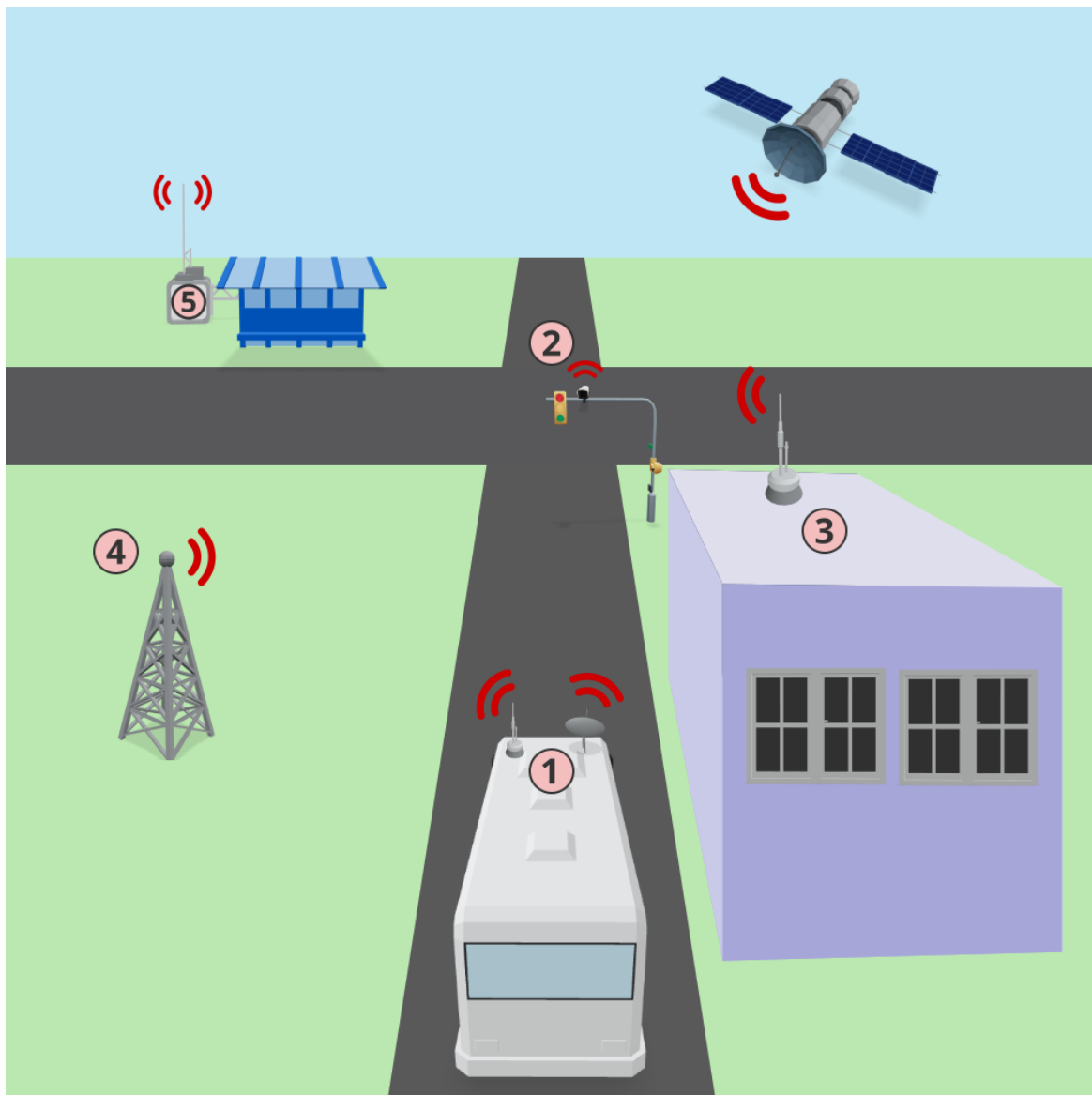
1. Járműre szerelt eszközök, mint a GPS, a kamera, a szenzorok és a kommunikációs eszközök.
2. Útmenti eszközök, mint a forgalomfigyelő kamerák és a jeladók
3. Vezérlőközpont, amely a járművek és az útmenti eszközök adatait dolgozza fel valós időben és végez számításokat ezek alapján.
4. Kommunikációs hálózat, amely a járművek, az útmenti eszközök és a vezérlőközpont között biztosítja az információ áramlását.
5. Megállóban felszerelt információ jelző eszközök, amelyek az utasokat tájékoztatják a járatok állapotáról és a menetrendről.

²Bus Fleet Management

³Real Time Information

⁴Automatic Vehicle Location

Az ITS rendszert és a benne lévő komponenseket szemlélteti a 3.1 ábra, ahol az egyes elemek számozása megegyezik az előbbi felsorolásban használt számozással.



3.1. ábra. Intelligens Közlekedési Rendszer

A buszok haladásának követésére többféle módszer is létezik, amelyek közül a legelterjedtebb a GPS alapú rendszerek használata, amelyben a buszokon levő GPS modulok határozzák meg a tartózkodási helyet és valamilyen más vezeték nélküli kommunikációs csatornán keresztül továbbítódik az adat a központi szerver felé. Sajnos a GPS működését számos tényező befolyásolhatja, mint az

időjárás, vagy a műholdjel gyenge minősége. Erre jelenthet megoldást egy új technológia, amely Bluetooth Low Energy (BLE) alapú jeladókat használ, amely a GPS-hez képest pontosabb helymeghatározást tesz lehetővé, de a hatótávolsága korlátozott. Ennek lényege hogy a buszokon csak egy kis fogyasztású Bluetooth jeladó működik, amely a megállóban lévő vevőkkel kommunikál, és ezek a vevők továbbítják az adatokat a központi szerverre. Ezzel a módszerrel a buszok helyzete pontosabban meghatározható, mint a GPS alapú rendszerekben, de a hatótávolság miatt csak a megállóban lehet használni. További nehézséget jelenthet, hogy minden megállóban fel kell szerelni egy jelfeldolgozó egységet, amelyhez áramforrás is szükséges. Nagyvárosokban ez még megoldható lenne, viszont távolsági buszjáratok esetében, ahol a megállók között nagy a távolság és nincs áramforrás minden vidéki megállóban, ott a GPS-es megoldás a legoptimálisabb [6].

Ilyen rendszerrel rendelkezik néhány nagyváros tömegközlekedése is, mint például a Londoni iBus, amely GPS alapú helymeghatározással és GPRS alapú adatátvitellel működik. Az iBus lehetővé teszi több mint 8000 busz valós idejű követését 30 másodperces frissítési rátával, amelyek adataiból kinyert információkat a buszokon található kijelzőkön és a megállóban is megjeleníti [7].

A jelenleg elérhető rendszereken még mindig lehetne fejleszteni, mert a más területek technológiai fejlődéséhez viszonyítva még mindig le van maradva a tömegközlekedés. Nagyobb a baj a kisvárosokban, ahol csak kis mértékben vagy egyáltalán nincs digitalizálva a rendszer. Ennek a helyzetnek a javításához szükség lenne egy olyan rendszerre, amely valós idejű információkat szolgáltat a járatokról, lehetővé teszi az online jegyvásárlást és az útvonaltervezést. Ehhez a buszok, vonatok és metrók különböző eszközökkel való felszerelésére van szükség, mint a jegyszkennelő automata, a kártyás fizetést biztosító POS terminál vagy a GPS modul. Egy ilyen rendszerrel való interakcióba lépéshez szükség van egy felhasználói felületre is, amely lehetővé teszi az utasok számára a különböző szolgáltatások elérését: az útvonaltervezést, a valós idejű forgalmi adatok megtekintését és a jegyvásárlást [8]. Erre a célra már léteznek megoldások, de ezek nem fedik le teljesen a busztársaságok igényeit és még messze állunk attól, hogy ezt minden város és busztársaság használni tudja.

3.1. Létező megoldások

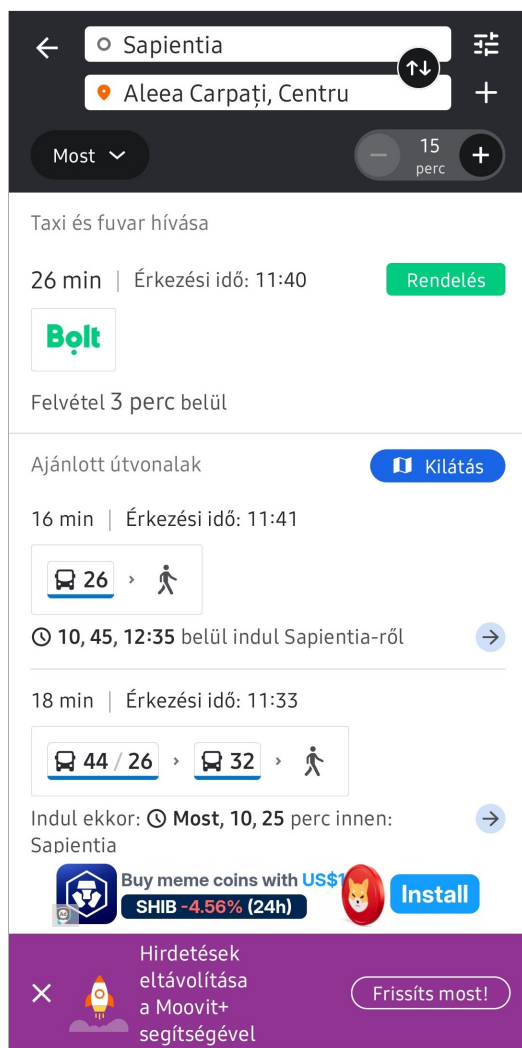
Kutatásunk során többek között rátaláltunk a Moovit és a Budapest Go nevű alkalmazásokra, amelyek két különböző megközelítést alkalmaznak a városi közlekedés digitalizálására, de céljaink hasonlóak: hogy a rendszer egyetlen platformon biztosítsa a tömegközlekedéshez kapcsolódó különféle szolgáltatásokat, beleértve a jegyvásárlást, útvonaltervezést és a valós idejű járatinformációk megjelenítését.

3.1.1. Moovit

A Moovit egy népszerű alkalmazás, amely a világ számos országában és városában működik, így egy szinte mindenhol elérhető szolgáltatást nyújt, viszonylag sok funkcióval. Az alkalmazás lehetővé teszi az utasok számára, hogy megtervezzék az útvonalukat, figyelemmel kísérik a járatokat és megvásárolják a jegyeket⁵. A közlekedési információkat az utasok okostelefonjaikon futó alkalmazások biztosítják, amelyek időnként beküldik a GPS pozícióikat a Moovit szervereire. Ezenkívül a felhasználók manuálisan is megadhatnak információkat, például a járatok késéséről vagy pillanatnyi telítettségéről. A rendszer ezeket az adatokat dolgozza fel és jeleníti meg a többi felhasználó számára, amennyiben ezek elegendő mennyiségben rendelkezésre állnak. Ezzel csak az a probléma hogy kevés országban éri el a felhasználók száma az elvárt szintet, így ezek a funkciók nem mindenhol érhetőek el [8].

Sajnos a kevés felhasználót számláló országok közé tartozik a miénk is, így csak a hivatalosan megadott adatokat jeleníti meg a Moovit, a felhasználóktól érkezőket nem, emiatt nálunk eléggé rövid az alkalmazás által nyújtott szolgáltatások listája. Marosvásárhelyen például csak a városi buszok menetrendje és útvonala érhető el, de ehhez sem társul élő forgalmi információ és jegyvásárlási lehetőség sem. A 3.2 ábrán látható egy képernyőkép a Moovit alkalmazásról.

⁵Moovit: <https://moovit.com>



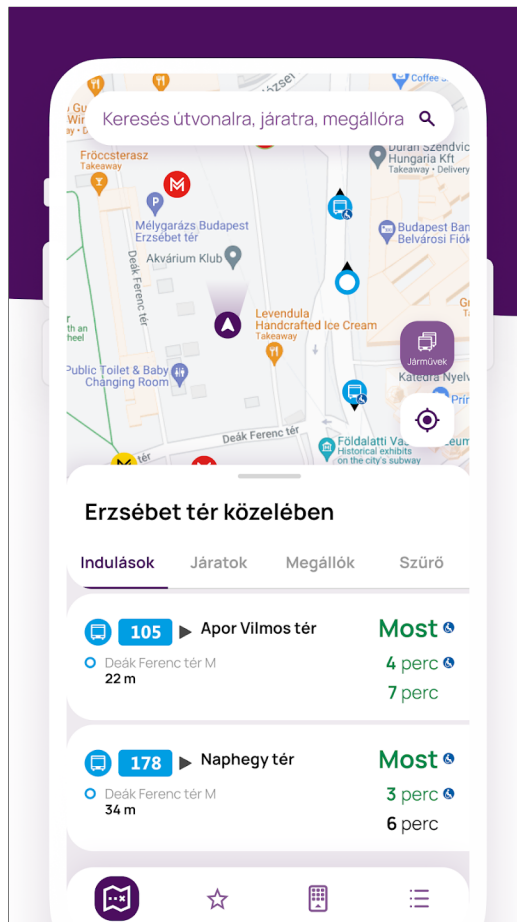
3.2. ábra. Moovit alkalmazás

3.1.2. Budapest Go

A másik alkalmazás a Budapest Go, amely egy általános közlekedési rendszer és a városi közlekedés minden elemét lefedi, így nem csak a buszokkal való közlekedésre használható, hanem majdnem az összes tömegközlekedési eszközzel a metrótól, a villamoson át, a hajóig, sőt az egyéni közlekedés megkönnyítése végett a gyalogos- és biciklis útvonalak is megtalálhatóak benne. Ez a megoldás talán közelebb áll a mi elképzeléseinkhez a funkcionalitás szempontjából, mert ugyanúgy biztosítja a jegyvásárlás lehetőségét, az útvonaltervezést és a valós idejű járatinformációkat, mint a Moovit, de a Budapest Go esetében ezek a funkciók sokkal részletesebbek és szélesebb körűek, emellett pedig

rengeteg egyéb funkciót is biztosít. A 3.3 ábrán látható a Budapest Go alkalmazás kezdőképernyője.

Egy lényeges probléma a Budapest Go-val az, hogy nem használható fel univerzálisan, mert ez a Budapesti Közlekedési Központ (BKK) saját rendszere, ezáltal pedig Budapestre van korlátozva és csak a helyi utasok tudják kihasználni az általa biztosított szolgáltatásokat⁶. Ezzel szemben mi a nyitottságra szeretnénk fektetni a hangsúlyt és egy olyan rendszert hoznánk létre, amelyet bármely busztársaság tudna használni, helytől függetlenül.



3.3. ábra. Budapest Go alkalmazás ⁷

⁶BKK x Supercharge: All-in-one mobility solution: <https://supercharge.io/work/details/budapest-go>

⁷Budapest Go - Google Play Áruház: <https://play.google.com/store/apps/details?id=hu.webvalto.bkkfutar>

4. fejezet

A rendszer specifikációi

A rendszer két fő részre bontható fel: háttérfolyamatok és megjelenítés. Mivel én a megjelenítés résszel foglalkozom, amely magában foglalja a felhasználói weboldalt, az adminisztrátori weboldalt és a mobil applikációt, ezért a következőkben csak ezekre a részekre vonatkozó követelmények specifikációját fogom részletezni. A háttérfolyamatok követelményei a kollégám dolgozatában lesznek ismertetve [1].

4.1. Felhasználói követelmények

A felhasználói követelmények azokat a funkciókat írják le, amelyre a felhasználó szemszögéből szükség van. A szoftver felületeinek általános követelményei a következők:

- A felület legyen logikusan felépítve, hogy felhasználó gyorsan és egyszerűen tudjon navigálni az oldalak között.
- Minden felhasználói bevitel esetén a rendszernek biztosítani kell a kényelmes használatot, valamint a megadott adatok helyességéről visszajelzést kell biztosítani beszédes hibaüzenetek megjelenítése által.
- A rendszernek reszponzívnak és hibatűrőnek kell lennie, azaz minden eszközön jól használhatónak kell lennie.

- A felhasználói felületnek egységesnek kell lennie, azaz minden platformon ugyanazok a színek és felületi elemek jelennek meg, hogy a felhasználók ne érezzék zavarónak az eszközök közötti váltást.
- Az ikonok és a gombok egyértelműen tükrözzék a funkciójukat, hogy a felhasználók könnyen megtalálják a keresett funkciókat.
- A felhasználói felületnek gyorsnak kell lennie, azaz a kérésekre azonnal válaszolnia kell, hogy a felhasználók ne érezzék lassúnak a rendszert.

A rendszer két különböző felületet tartalmaz: felhasználói- és adminisztrátori felület, ezek pedig különböző felhasználói követelményekkel rendelkezhetnek, így ezeket külön-külön fogom részletezni.

4.1.1. Felhasználói felület

A felhasználói felület funkcióinak nagy része bejelentkezés nélkül használható. A fiókot igénylő funkciók a jegyvásárlás és a jegyek listázása. Az ezek eléréséhez szükséges regisztráció és bejelentkezés bármely oldalról elérhető. A felhasználói felület követelményei a következők:

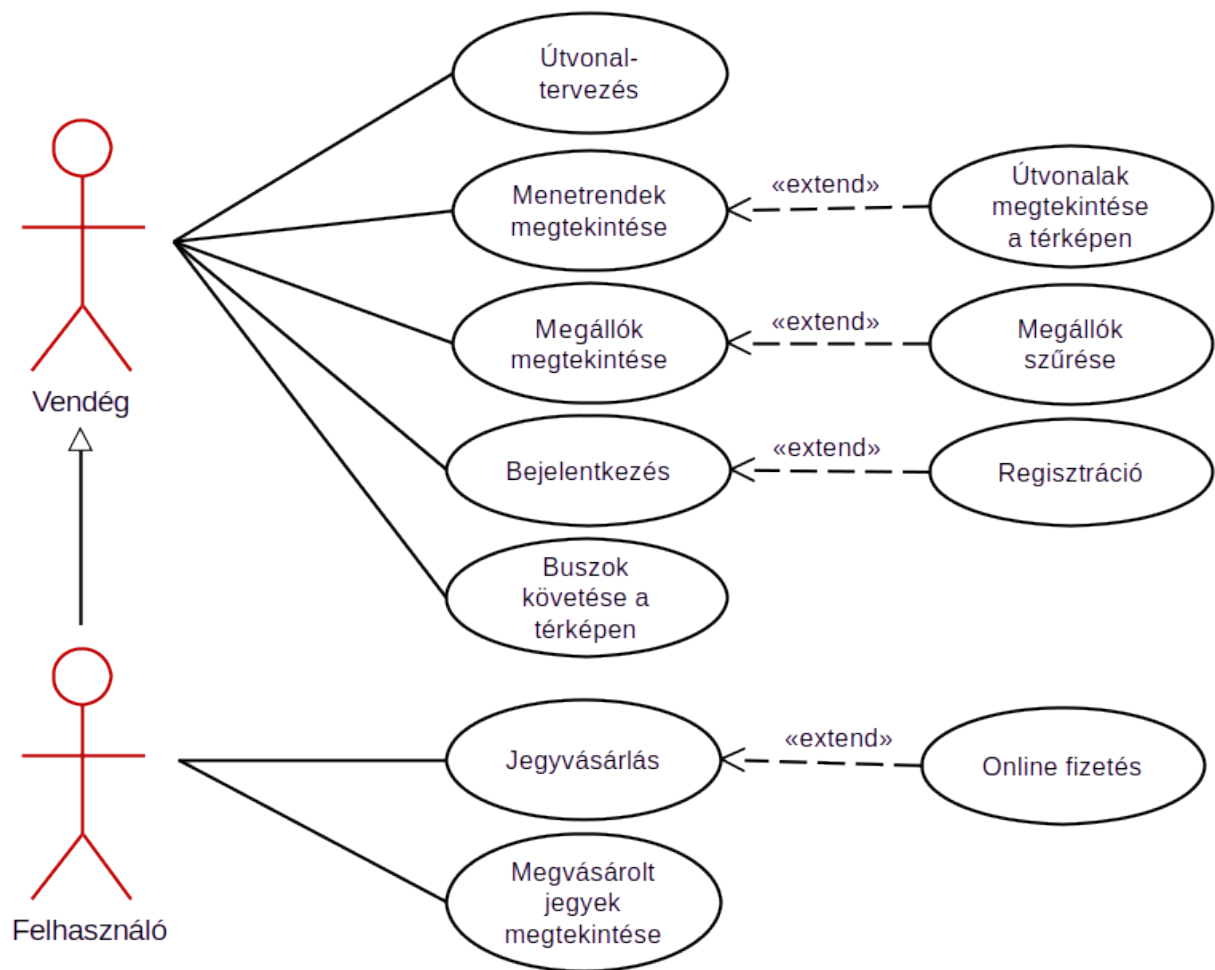
- Legyen lehetőség utazást tervezni dátum, idő, kezdő- és célállomás alapján
- A talált járatok jelenjenek meg indulási idő szerinti növekvő sorrendben
- Legyen lehetősége a bejelentkezett felhasználónak jegyet vásárolni online a járatokra
- A rendszer jelenítse meg a buszok és megállók menetrendjeit
- A járatok útvonalait lehessen megtekinteni a térképen
- Térképen megtekinthetők legyenek a megállók és szűrni is lehessen közöttük település vagy buszjárat alapján
- A rendszer biztosítsa a buszok valós idejű pozícióinak követését a térképen
- A megvásárolt jegyek megtekinthetők legyenek bejelentkezés után
- A jegyek QR-kódokkal legyenek ellátva a könnyebb beolvasás érdekében

4.1.2. Adminisztrátori felület

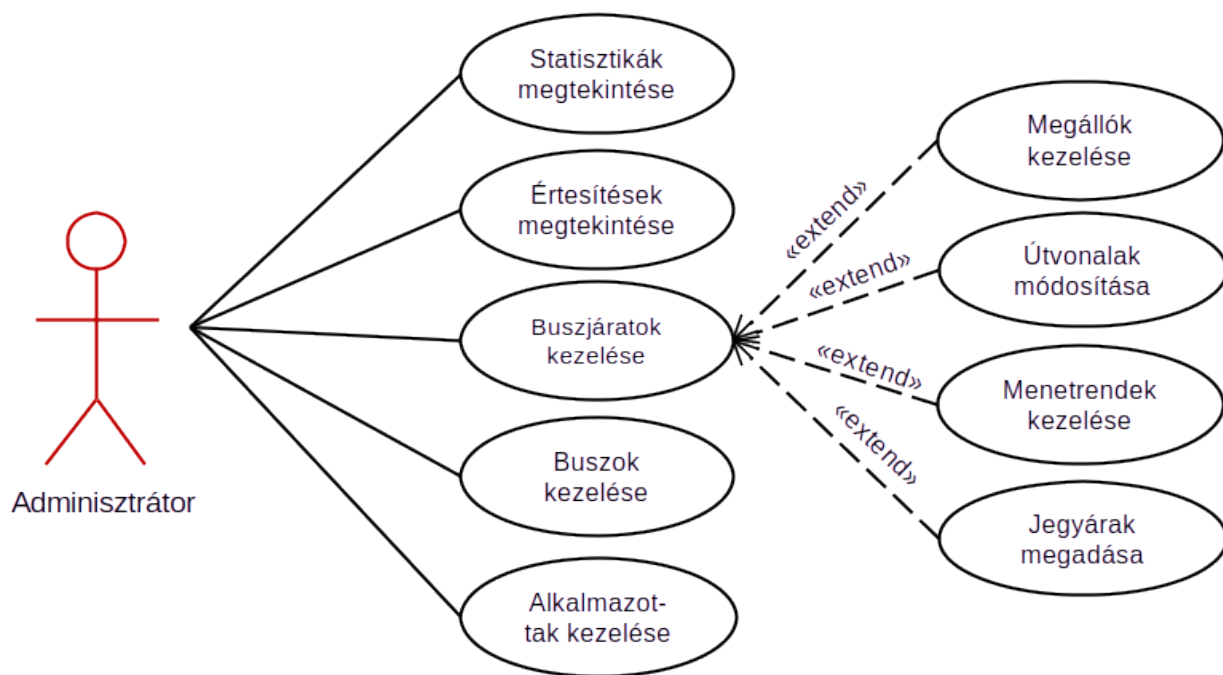
Az adminisztrátori felület kizárólag bejelentkezéssel elérhető, új fiókokat pedig csak a cégek főnökei tudnak létrehozni, mindenki a saját busztársaságához. Itt regisztrációs lehetőség nincs, így meg lehet akadályozni az adminfelülethez való illetéktelen hozzáférést. A társult vállalatok főnökei a partnerség kezdetekor megkapják az admin jogú fiókjaikat, amelyekkel hozzáférnek a felülethez. Az adminfelületre vonatkozó követelmények:

- A főoldalon statisztikák jelenjenek meg az eladott jegyek számáról, a megtett kilométerről és a bevételről, időintervallumok szerint csoportosítva
- A rendszer jelenítse meg a felületen a fontos emlékeztetőket és értesítéseket
- Lehessen megtekinteni a megállók listáját, új megállókat hozzáadni térképen való megjelölés segítségével, valamint a meglévő megállókat szerkeszteni vagy törölni
- A rendszer listázza a buszjáratokat és biztosítson lehetőséget új járat létrehozására, meglévő járat szerkesztésére, valamint járatok törlésére
- A járatokhoz lehessen csatolni és eltávolítani előre létrehozott megállókat
- A járatok indulási időpontjait jelenítse meg a rendszer, ezek változtatásait és törlését pedig mentse el az adatbázisban
- A buszjáratokhoz és az útvonalak különböző részeihez lehessen külön-külön jegyárat megadni
- A rendszer jelenítse meg a buszok listáját és biztosítson lehetőséget új busz hozzáadására, meglévő busz szerkesztésére, valamint buszok törlésére
- A buszokhoz lehessen megadni utadó, biztosítás és műszaki vizsga lejáratú időpontokat
- A buszokhoz hasonlóan lehessen kezelni az alkalmazotti fiókok listáját is, létrehozással, szerkesztéssel és törléssel

A rendszer funkcionalitását leíró használati eset diagramok a 4.1 és 4.2 ábrákon láthatóak.



4.1. ábra. Use-case diagram a végfelhasználók lehetőségeiről



4.2. ábra. Use-case diagram az adminisztrátorok lehetőségeiről

4.2. Rendszerkövetelmények

4.2.1. Funkcionális követelmények

A funkcionális követelmények a felületen megjelenő funkciók működését írják le. Ezek foglalják össze, hogy adott bemenetre milyen kimenetet kell szolgáltatnia a rendszernek. A Bus4U esetében ezeket is két csoportba sorolhatjuk: felhasználói és adminisztrátori követelmények.

A felhasználói weboldalon és a mobilalkalmazásban bejelentkezés nélkül hozzá lehet férni a főoldalhoz, a menetrendekhez, a megállókhoz és az élő térképhez. Az egyetlen oldal, ami nem jeleníthető meg azonosítás nélkül az a jegyek oldal, innen a rendszernek át kell irányítania a bejelentkezési felületre. Itt biztosítani kell a regisztráció lehetőségét is, amennyiben a felhasználó még nem rendelkezik fiókkal. A regisztráció során a rendszer le kell ellenőrizzen minden megadott adatot, még az email tulajdonjogát is, azáltal hogy egy ellenőrző kódot küld a megadott email címre, amelyet be kell ír-

ni a felületen megjelenő szövegmezőbe. Sikeres regisztráció vagy bejelentkezés után a rendszernek minden adatot biztonságosan el kell mentenie, a jelszavat hash-elt formában és át kell irányítania a felhasználót a főoldalra.

A főoldalon lévő szövegmezőkben megadott adatok alapján a rendszernek le kell kérnie a szerverről néhány lehetséges járatot, amelyek indulnak az adott településről, adott időben, a megadott célállomásra. A megjelenített találatokhoz tartoznia kell egy jegyvásárlási lehetőségnek is, amelyben meg kell adni a jegyek számát és véglegesíteni kell a vásárlást. A jegyvásárlás is bejelentkezést igényel, így az azonosítatlan felhasználót a rendszer vásárlás előtt átirányítja a bejelentkezéshez. A vásárlás csak az erre kijelölt oldalon történő kifizetés után véglegesíthető, ezt követi a jegy kódjának kigenerálása és elmentése. Ezek után a felhasználók megtekinthetik a megvásárolt jegyeket a Jegyek oldalon. A menetrendek oldalon a felhasználóknak lehetőségük van a buszok és megállók közötti választásra lenyíló listákból, amelyet követően a rendszer kilistázza az adott járat megállóhoz kötött időpontjait egy táblázatban. Ugyanitt a rendszernek lehetőséget kell biztosítania a buszok útvonalának megjelenítésére térképen. A megállók oldalon a felhasználóknak lehetőségük van az összes megálló megtekintésére térképen, valamint lenyíló listákat használva, a megállók szűrésére település vagy buszjárat alapján is. A térképnek interaktívnak kell lennie, így a megjelölt megállókra való kattintás után meg kell jelenítenie az adott megálló adatainak egy kis előugró ablakban. Az élő térkép oldalon a felhasználóknak lehetőségük van a buszok valós idejű pozícióinak követésére térképen. A megjelenített buszokat a rendszernek valós időben kell frissítenie, legalább félpercenként, hogy a felhasználók pontos információkat kapjanak a buszok helyzetéről. Az összes térképes oldalon a felhasználóknak lehetőségük van a térkép nagyítására és kicsinyítésére, valamint a térkép mozgatására is, egérrel vagy érintéses gesztusok segítségével. A felhasználónak meg kell tudnia tekinteni a saját pozícióját is a térképeken, amennyiben engedélyezte a helymeghatározáshoz való hozzáférést.

Az adminisztrátori weboldalon szükség van azonosításra, anélkül a rendszer minden oldalról átirányít a bejelentkezéshez. Itt a regisztráció nem lehetséges, így külső, admin szerepkörrel nem rendelkező felhasználók nem tudnak hozzáférni az adminisztrátori felülethez. A főoldalon található statisztikák között szerepel az eladott jegyek száma, a megtett kilométerek száma és a bevétel összege, időintervallumok szerint csoportosítva, táblázatos formában. Mivel minden busztársaságnak el kell legyenek különítve az adatai, ezért a szerver mindig a bejelentkezett adminhoz tartozó cég adataiból számolja

ki a statisztikákat. A főoldalon a rendszernek meg kell jelenítenie az esetleges értesítéseket is, előugró "Toast" üzenetek segítségével. A megállók oldalán az adminisztrátoroknak lehetőségük van a megállók listázására, az ezekre való egyenkénti kattintás után pedig a szerkesztésükre is. Emellett megállók törlésére és új megállók hozzáadására is lehetőség van egy interaktív térkép segítségével. A térképre kattintva a hely koordinátáit a rendszer beilleszti az űrlapba, ahol meg kell adni még a megálló nevét és a címet is. Utóbbit, amennyiben elérhető, le kell kérnie a rendszernek a térképszolgáltatótól, hogy ezzel is gyorsítsa a létrehozást. A járatok oldalán az adminisztrátoroknak lehetőségük van a buszjáratok kiválasztására egy lenyíló menüből, ezt követően pedig meg kell jelennie a járatszerkesztő űrlapnak. Ugyanitt lehetőség lesz járatok hozzáadására és törlésére is. A menetrendek oldalán az adminisztrátoroknak lehetőségük van a járatok indulási időpontjainak és jegyárainak megtekintésére, szerkesztésére és törlésére. Az órarendek bevitelét le kell lehessen bontani napokra, ezeket pedig dinamikusan megjeleníteni, hogy minél kevesebb helyet foglaljon az oldalon. A buszok és alkalmazottak oldalakon a rendszernek biztosítani kell egy egységes felületet, amelyben lehetőség nyílik az elemek listázására, új elemek hozzáadására, meglévő elemek szerkesztésére és ezek törlésére is. A buszoknál fontos megadni az útdó, biztosítás és műszaki vizsga időpontjait, mert a rendszer ezek alapján fog emlékeztetőket megjeleníteni a felületre való belépések alkalmával. Az alkalmazottaknál meg kell adni a nevet, a beosztást és a telefonszámot, valamint az emailt és a jelszót is, amelyet a rendszer hash-elt formában tárol. Utóbbiak az alkalmazott bejelentkezéséhez lesznek szükségesek, így a rendszernek ellenőriznie kell őket.

4.2.2. Nem funkcionális követelmények

Szervezéssel kapcsolatos követelmények

- Programozási nyelvek: Dart, HTML, CSS, JavaScript
- Keretrendszerek: Flutter, Vue.js
- IDE: Visual Studio Code
- Verziókövetés: Git és GitHub
- Webhost: Netlify

- Csoportos kommunikáció: Slack

Termékkel kapcsolatos követelmények

A felhasználói weboldal és az admin felület használatához egy modern, internetezésre alkalmas eszközre van szükség, a térképfunkciók teljes kihasználásához engedélyezni kell a helymeghatározást. Ajánlott a 2018 után kiadott böngészőverziók használata:

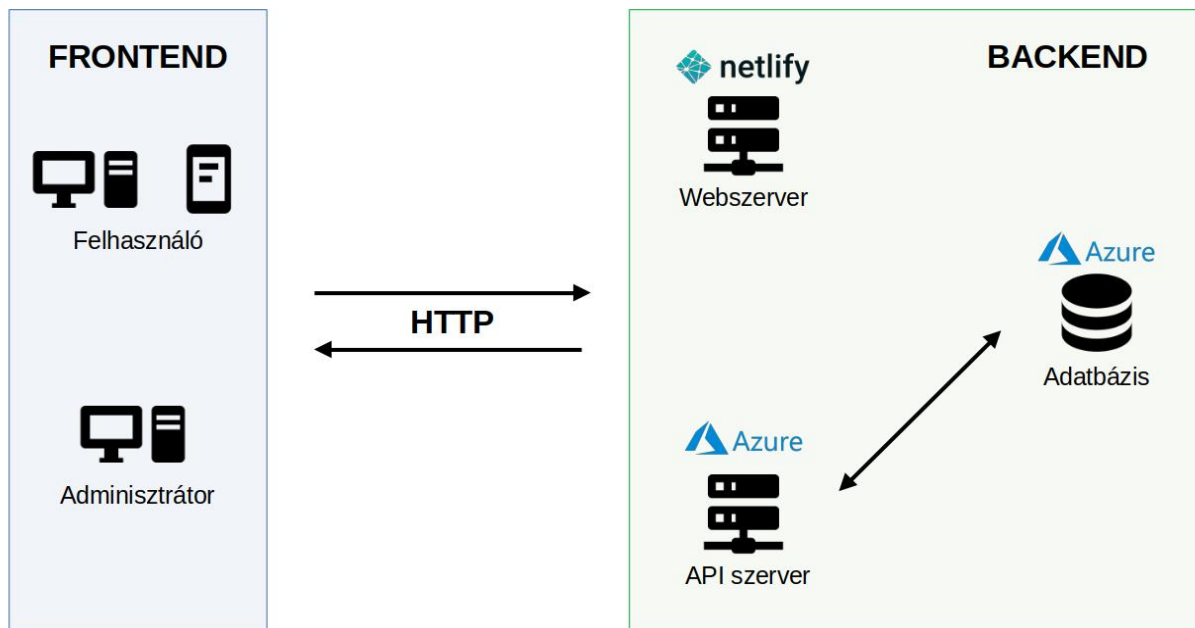
- Chrome ≥ 87
- Firefox ≥ 78
- Safari ≥ 14
- Edge ≥ 88

Az mobilapplikáció használatához szükség van egy okostelefonra vagy tabletre, amelyen minimum Android 5.0 vagy iOS 8.0 operációs rendszer fut. Fontos hogy legyen rajta internet elérés (3G,4G,5G vagy Wi-Fi), legalább 120MB szabad tárhely és 516-1024 MB szabad memória. Itt is ajánlott a helymeghatározás (GPS) engedélyezése, hogy a térképfunkciók helyesen működjenek.

5. fejezet

A rendszer architektúrája

Az általunk tervezett rendszer architektúra diagramja a 5.1 ábrán látható. A rendszer frontend és backend része jól elkülöníthető egymástól, az egyes komponensek közötti kommunikáció titkosított HTTP kérések segítségével valósul meg. Először tárgyalom a frontend és a backend felépítését, majd a közöttük megvalósuló kommunikációt.

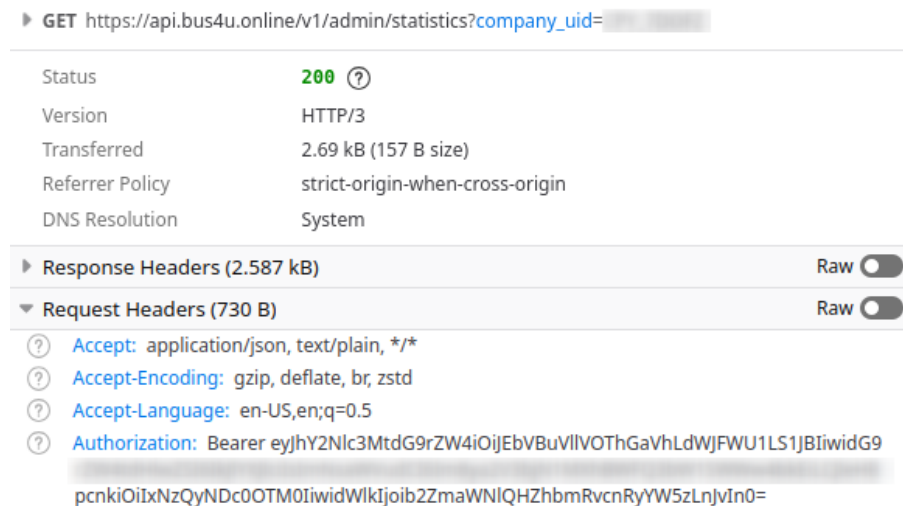


5.1. ábra. A rendszer architektúrája

5.1. Frontend

A frontend a felhasználói felületet biztosítja, amelyen keresztül az emberek interakcióba léphetnek a rendszerrel. Jelen esetben a frontend két weboldalból és egy mobilalkalmazásból áll. A két weboldal teljesen elkülönül egymástól, mivel az egyik az adminisztrátoroknak, míg a másik a végfelhasználóknak készült. A weboldalak működése eltér a klasszikustól, mivel ezek egyoldalas alkalmazások (Single Page Application), így kliens oldali feldolgozást használnak, azaz a felhasználói felületet a kliens oldalon generálja a böngésző, nem pedig a webszerverről érkezik készen. Jelen esetben a webszerver csak a statikus fájlokat szolgáltatja, a dinamikus tartalmakat a kliens oldali JavaScript illeszti be az oldalakba. Ebből a szempontból a weboldal és a mobilalkalmazás működése megegyezik, mivel mindkettő direkt módon az API szerverrel kommunikál, REST kéréseken keresztül. Az adatok JSON típusú szöveggént közlekednek a végpontok között az egyszerű feldolgozhatóság végett.

Fontos megemlíteni, hogy egyes API végpontokhoz azonosítás szükséges, ami úgynevezett JWT¹ tokenek segítségével valósul meg. Úgy a weboldalak, mint az alkalmazás eltárolják a szervertől kapott tokenet és hozzáfűzik minden API kéréshez, ezzel azonosítva a felhasználót. Erre egy példa a 5.2 ábrán látható, ahol az adminisztrátorok számára megjelenített statisztikát kéri le a rendszer a backendtől. Ez a kérés egy Authorization fejléccel van ellátva, amely tartalmazza a JWT tokenet.



5.2. ábra. Azonosítást tartalmazó API kérés

¹JSON Web Token

Egyes oldalakon térképek is megjelennek, amelyeket a web esetében a MapBox, míg a mobil esetében a Google Maps szolgáltat. Az eltérő technológiák használatának oka, hogy a MapBox olcsóbb és testreszabhatóbb, így weben ez volt az ideális, viszont mobilon, Flutterrel kombinálva (ami Google termék) jobb integrációja volt a Google Maps-nek. Emiatt mindkét technológiára szükség volt, hogy a lehető legjobb felhasználói élményt biztosítsam a különböző platformokon.

Az online fizetések kezelésére a Stripe² szolgáltatást használtam, mert ez egy biztonságos és egyszerű fizetési megoldás, amelyet könnyen lehet integrálni minden platformon. A Stripe SDK rengeteg programozási nyelven elérhető, így JavaScript nyelvhez az NPM³ csomagkezelő segítségével is telepíthető, valamint a Flutterben is elérhető egy pub.dev csomag formájában. A weboldalon való fizetéshez a Stripe beágyazott fizetési felületét alkalmaztam, amely lehetővé teszi a felhasználók számára, hogy biztonságosan fizessenek anélkül, hogy elhagynák a Bus4U oldalát.

5.2. Backend

A backend a szerveroldali logikát és az adatbázis elérést biztosítja. A weboldalak API hívásokon keresztül kapják meg az adatokat és dinamikusan jelenítik meg őket, ezért nincs egy olyan webszerver, amely backend oldalon dolgozna ki a weboldalakat, csak egy olyan, amely statikus fájlokat szolgáltat. Emiatt a webszerver nem tartalmaz üzleti logikát és az adatbázissal sincs összeköttetésben.

Ezzel szemben van egy API szerver, amely az összes felhasználói felületet kiszolgálja adatokkal. Az ő feladata az adatbázissal való kommunikáció, az adatok feldolgozása és az ezek elérhetővé tétele a frontend számára, API végpontok formájában. Ez a komponens kezeli a felhasználóktól érkező kéréseket is, amelyek az adatbázist szeretnék módosítani. Minden ellenőrzés, biztonsági vizsgálat is itt folyik le, hogy a frontend csak a megfelelő adatokhoz férhessen hozzá és csak azokat módosíthassa.

Az adatok kiszolgálásának folyamatában az adatbázisnak csak az a feladata, hogy a felhasználók, buszok, járatok, megállók és jegyek adatait biztonságosan tárolja és azokat az API szerver számára elérhetővé tegye olvasásra és módosításra is.

Részletesebb leírást a backend felépítéséről és működéséről a másik dolgozatban lehet találni [1].

²Stripe: <https://stripe.com/en-ro>

³Node Package Manager: <https://www.npmjs.com/>

5.3. Kommunikáció

A felek közötti kommunikáció REST API hívásokkal valósul meg, ezt szemlélteti az 5.4 ábrán megjelenített szekvencia diagram, amely a felhasználó által megvásárolt jegyek listázásának folyamatát írja le. Amikor a felhasználó a frontend felületen keresztül a My Tickets oldalra navigál, egy GET kérés indul el a szerver felé, amely tartalmazza a felhasználó adatait. A backend először ellenőrzi a kérést, majd a megfelelő jegyeket lekérdezi az adatbázisból és visszaküldi a frontendnek, ahol azok idő szerinti rendezés után megjelennek a felhasználó számára. A még érvényes jegyekhez generál a rendszer egy QR kódot is, hogy könnyű legyen majd innen beolvasni. Amennyiben valahol hiba történik, a rendszer informálja erről a felhasználót. A következő kódrészlet intézi a kérést a frontend oldalról:

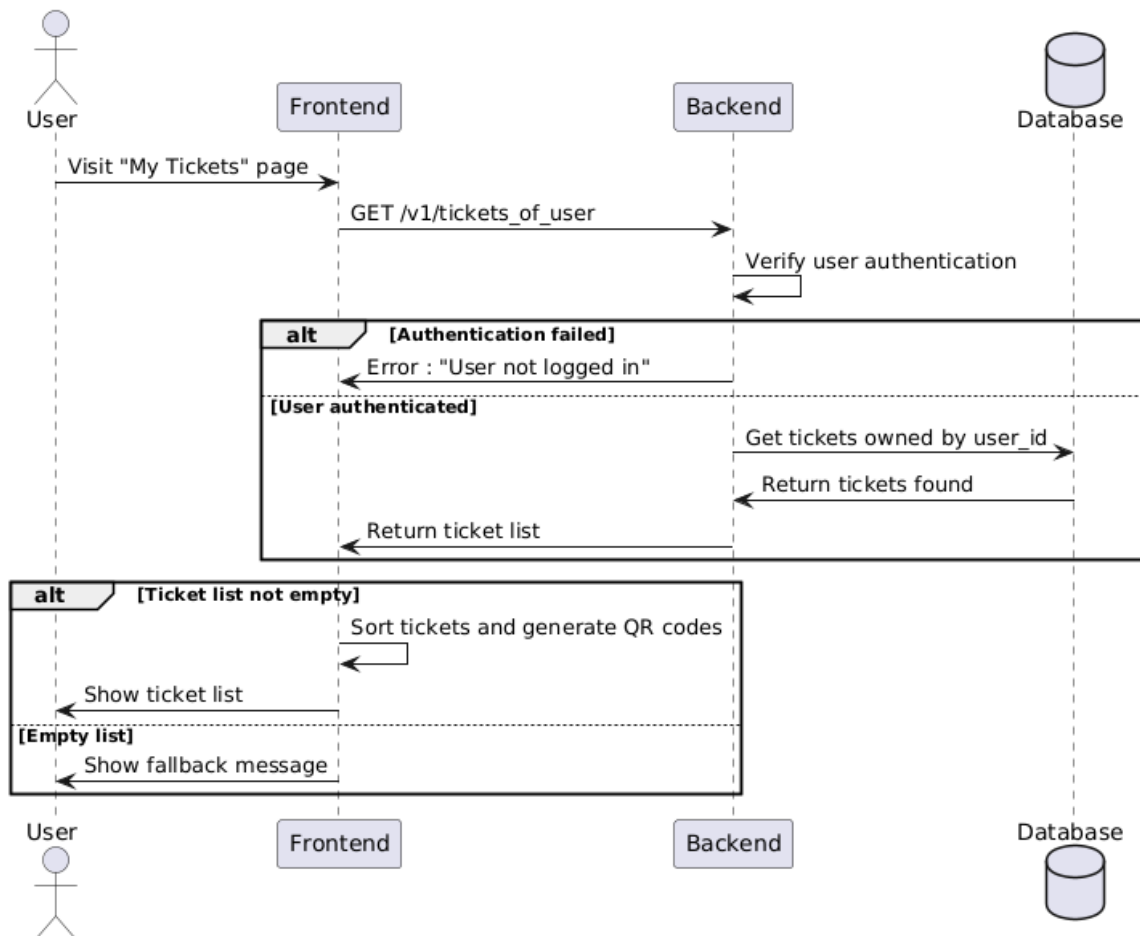
```
isLoading.value = true
axios.get('https://api.bus4u.online/v1/tickets_of_user', { headers: { Authorization: user.
  authorization }, params: { uid: user.uid }})
  .then(rsp => {
    isLoading.value = false
    tickets.value = rsp.data.sort((a, b) => {
      if(!a.is_valid && b.is_valid) return 1
      if(!b.is_valid && a.is_valid) return -1
      return b.date_of_purchase.localeCompare(a.date_of_purchase)
    })
    for(let i=0; i<tickets.value.length; i++) {
      if(!tickets.value[i].is_valid) continue
      QRCode.toDataURL(tickets.value[i].ticket_uid, { width: 250 }).then(rsp => tickets.value[i].qr =
        rsp)
    }
  }).catch(() => {
    tickets.value = []
    isLoading.value = false
  })
```

Egy ilyen kérésre, a backend által adott válasz az 5.3 ábrán látható.

JSON

- ▼ 0: Object { ticket_uid: "TIC_ECB5K", date_of_purchase: "2024-10-11T17:25:26.981+03:00", expiration_date: "2024-11-11T16:25:26.981+02:00", ... }
- ticket_uid: "TIC_ECB5K"
- date_of_purchase: "2024-10-11T17:25:26.981+03:00"
- expiration_date: "2024-11-11T16:25:26.981+02:00"
- from_station_uid: "STT_EV9LM"
- to_station_uid: "STT_PXVPC"
- from_station_name: "Izvor"
- to_station_name: "Grand"
- is_valid: true
- route_uid: "ROU_SOXDN"
- route_name: "26"
- ticket_price: "200.0"
- 1: Object { ticket_uid: "TIC_SK8EA", date_of_purchase: "2024-10-11T17:25:27.069+03:00", expiration_date: "2024-11-11T16:25:27.069+02:00", ... }
- 2: Object { ticket_uid: "TIC_AG9KQ", date_of_purchase: "2024-10-11T17:29:37.005+03:00", expiration_date: "2024-11-11T16:29:37.005+02:00", ... }

5.3. ábra. A jegyek listázásának válasza



5.4. ábra. A jegyek listázásának folyamata

Ugyanígy leírható a regisztráció folyamata is, ahol a felhasználó megadja a nevét, az email címét és az új jelszót, majd a rendszer az adatok ellenőrzése után elküldi egyelőre csak az email címet a backend felé. A backend az email szerver segítségével küld egy 4 számjegyű kódot a felhasználó által megadott email címre. Ezután a felhasználó be kell írja ezt az ellenőrző kódot a felületen ennek megfelelő helyre. Ezt a kódot a backend ellenőrzi és amennyiben helyes, a frontend elküldi a felhasználó további adatait is, amelyet a backend elment az adatbázisban. Ha minden lépés sikeresen végrehajtódik, a frontend elmenti a bejelentkezési adatokat kliens oldalon is és átirányítja a felhasználót a főoldalra. Ellenkező esetben hibaüzenet jelenik meg a lépéssorozat azon pontján, ahol a hiba keletkezett. A regisztráció utolsó lépését (felhasználói adatok elküldése) megvalósító kódrészlet a következő:

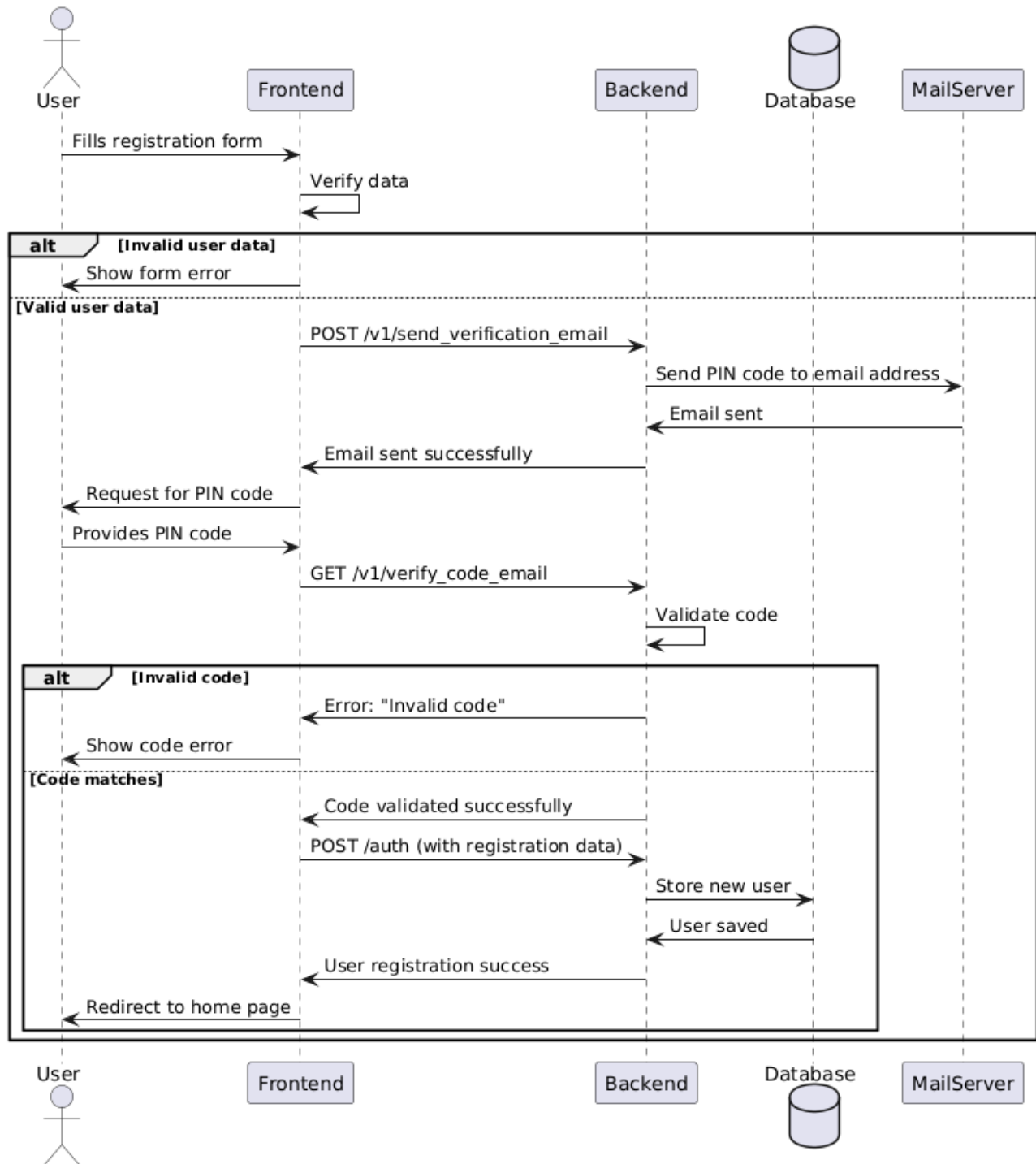
```
async function sendForm() {
  axios.post('https://api.bus4u.online/auth', form).then((rsp) => {
    if(rsp.data.data.uid) {
      router.replace({ name: 'home' })
      user.signIn(rsp.data.data, rsp.headers)
    } else {
      isLoading.value = false;
    }
  }).catch((e) => {
    if(e.response) error.value = e.response.data.error;
    isLoading.value = false;
  });
}
```

JSON

```
status: "success"
data: Object { provider: "email", uid: "12345678901234567890@student.ms.sapientia.ro", allow_password_change: false, ... }
  provider: "email"
  uid: "12345678901234567890@student.ms.sapientia.ro"
  allow_password_change: false
  firstname: "John"
  lastname: "Doe"
  email: "12345678901234567890@student.ms.sapientia.ro"
  phone_number: null
  language: null
  stripe_id: "1234567890123456789012345678901234567890"
  created_at: "2025-02-19T16:48:25.272+02:00"
  updated_at: "2025-02-19T16:48:25.363+02:00"
```

5.5. ábra. A regisztrációra adott válasz

A regisztrációs adatok elküldésére adott válasz az 5.5 ábrán látható, míg a teljes regisztrációs folyamatot szemléltető szekvencia diagram az 5.6 ábrán látható.



5.6. ábra. A regisztráció teljes folyamata

5.4. Felhasznált technológiák

A felhasználói és az admin weboldalak Vue.js keretrendszerben készültek, Vuetify komponenseket felhasználva. Azért esett a választásom erre a kombinációra, mert a Vue által gyors és hatékony weboldalak lehet létrehozni, a Vuetify pedig felgyorsítja a fejlesztést előre elkészített, jól kinéző komponensek szolgáltatásával. A mobil platformokra szánt alkalmazás elkészítéséhez Flutter technológiát használtam, mert ennek segítségével egyetlen kódbázisból ki lehet generálni Android és iOS készülékekre is a natív alkalmazást, amely hatékonyan működik minden eszközön a keretrendszer optimalizációi miatt. Mindkét implementáció a Material Design irányelveit követi, mert ez egy szép és egységes felületet biztosít a felhasználók számára, bármilyen eszközön használják a Bus4U-t.

5.4.1. Vue.js

A Vue.js egy progresszív JavaScript keretrendszer, amelyet azért hoztak létre, hogy a webes felhasználói felületek fejlesztése egyszerűbb és gyorsabb legyen. A JavaScript keretrendszerek célja, hogy megkönnyítsék a webprogramozást, azáltal hogy előre megírt osztály- és függvénykönyvtárakat szolgáltatnak, amelyek segítenek lerövidíteni a fejlesztési időt. Első ilyen keretrendszer a jQuery volt, amely a DOM (Document Object Model) manipulációt és az AJAX (Asynchronous JavaScript And XML) hívásokat tette egyszerűbbé, majd később a React és az Angular keretrendszerek jöttek, amelyek a komponens alapú fejlesztést tették lehetővé. Ez a fejlődés viszont mind nagyobb és nagyobb kiterjedésű csomagokat eredményezett, ami kissé csökkentette a weboldalak teljesítményét. A Vue.js ezt a problémát próbálja megoldani, azzal hogy egy könnyű, gyors és hatékony keretrendszert kínál, amely egy Virtuális DOM modell segítségével gyorsítja a futást és amelynek a forráskódja akár 24 Kb-ra is zsugorítható, így viszonylag gyors a betöltése is [9].

A Vue.js két alapköve a deklaratív UI felépítés és a reaktivitás. A deklaratív felépítés azt jelenti hogy az alap HTML kódot a Vue kiegészíti egy saját sablon-szintaxissal, amely lehetővé teszi a változók beágyazását a HTML kódba. A reaktivitás azt jelenti, hogy amikor a HTML sablonban felhasznált JavaScript változók értéke megváltozik, akkor a Vue érzékeli ezt és automatikusan megváltoztatja a megjelenített HTML kódot, így mindig a legfrissebb adatokat látja a felhasználó. Ugyanez végbemegy fordított irányban is, mert a felhasználói interakciók eredményeképpen a Vue automatiku-

san frissíti a háttérben lévő változókat is, így lesz a kommunikáció a DOM és a Vue között kétirányú [9, 10].

A Vue keretrendszer egyik legnagyobb előnye, hogy könnyen tanulható és alkalmazható, így gyorsan lehet vele fejleszteni. Ennek egyik oka, hogy komponens alapú, amely lehetővé teszi a weboldalak felépítését újrafelhasználható, különálló komponensekből, amelyeket később össze lehet fűzni egy nagyobb alkalmazássá. Ezek úgynevezett Single-File Component-ek (egyfájlos-komponensek) formájában jelennek meg, amely azt jelenti, hogy az adott komponens HTML, CSS és JavaScript kódja mind egy fájlban belül található, ezzel egy átlátható és moduláris alkalmazásfelépítést eredményezve [10]. Emellett a Vue nagyon jó online dokumentációval rendelkezik, amelyben minden funkció részletesen le van írva, de különleges problémákkal és kérdésekkel a dinamikusan növekvő fejlesztői közösséghez is lehet fordulni, így mindig könnyen meg lehet találni az éppen szükséges információt és nagyon gyorsan el lehet sajátítani a keretrendszer használatát [9].

A Vue legújabb verziója a Vue 3, amely a komponensek leírására 2 féle API-t⁴ is biztosít: Options és Composition. A Composition API a 3-as verziótól került bevezetésre és különféle teljesítménybeli optimalizációkat tartalmaz a régi Options API-jal szemben, de a szintaxisa is egyszerűbb, mert közelebb áll az alap JavaScript szintaxisához [10]. A Vue 3 a TypeScript támogatását is bevezette, amely egy statikus típusellenőrző nyelv a JavaScript fejlesztéséhez és amely segít a hibák korai felfedezésében, valamint a kód minőségének javításában. Én a munkám során a Vue 3-at és a Composition API-t használtam, mivel ezek a legújabb és legelőnyösebb technológiák jelenleg.

Fontos megemlíteni a Vue ökoszisztémájába tartozó egyéb eszközöket is, amelyek segítik a fejlesztést. Ilyen például a Vue Router, amely a weboldalak közötti navigációt teszi lehetővé, a Pinia, amely a globális állapotkezelést segíti, a Vite, amely a projekt generálását és a fejlesztést segíti, vagy a Vitest amely a teszteléshez szolgálat hasznos eszközöket [10]. Ezek közül kiemelném a Vite nevű build tool-t (építő eszközt), amelynek a gyors újratöltés funkciója figyeli a forráskódban létrejött módosításokat és azonnal frissíti a weboldalt, így a fejlesztőnek nem kell manuálisan a frissítés gombot nyomogatni minden begépelte kódsor után.

A felhasználó szemszögéből a leglátványosabb tulajdonság, hogy nagyon gyors a betöltés és a navigáció, mivel a Vue Single Page Application üzemmódja és a Vue Router kombinációja által lé-

⁴Application Programming Interface

nyegében csak egyszer töltődik be a webalkalmazás, utána az oldalak közötti navigáció során csak a tartalom frissül, nem pedig az egész oldal, így lényegesen kevesebb adat közlekedik a hálózaton keresztül és a böngészőnek is kevesebb dolga van az oldalak megjelenítésénél [10]. Ezeknek a funkcióknak a segítségével egy gyors és hatékony felhasználói felületet lehet készíteni, amely a gyengébb hálózati lefedettséggel és a gyengébb hardverrel rendelkező eszközökön is jól használható.

5.4.2. Vuetify

A Vuetify egy Material Design alapú Vue.js-re épülő, nyílt forráskódú UI könyvtár, amely lehetővé teszi a gyors és egyszerű felhasználói felületek készítését, azáltal hogy rengeteg előre elkészített Vue komponenst tartalmaz, amelyeket egyszerűen be lehet integrálni a weboldalba, így sok időt lehet vele spórolni. A Vuetify segítségével gyorsan és hatékonyan sikerült felépítenem a weboldalak felhasználói felületeit, mi több, ezek jól néznek ki és jól is működnek minden tesztelt eszközön.

5.4.3. Flutter

A Flutter egy nyílt forráskódú, Google által kifejlesztett, cross-platform, UI készítő keretrendszer, amely lényege, hogy egyetlen kódbázisból ki lehet generálni az alkalmazásokat a legnépszerűbb platformokra. Ezek a platformok a következők: Android, iOS, Windows, macOS, Linux, valamint a web.

A Flutter a Dart programozási nyelvet használja, amely szintén a Google fejlesztése. Ez egy modern, típusos, objektum orientált nyelv, amelyet könnyen és gyorsan el lehet sajátítani. Eredetileg a Dart a JavaScript lecserélésére volt létrehozva a webfejlesztésben, ami eddig nem tudott valósulni, viszont idővel a Flutter technológia alapjává vált és így tett szert népszerűsége. A Flutter egyik nagy előnye, hogy egy widget rendszert használ, amely lehetővé teszi a felhasználói felületek egyszerű és gyors elkészítését, valamint a különböző platformokra történő kiadást. Ezek a widgetek alapértelmezetten a Material Designra épülnek, így egy egységes, intuitív és a felhasználót megragadó felület építhető fel belőlük [11], amely összhangban van a Vuetify webes komponenseivel is. A Flutter egy másik nagy előnye, hogy a natív alkalmazásokhoz hasonlóan gyors és hatékony, mivel natív kódra fordítja le a Dart kódot, így a lehető legjobb teljesítményt nyújtja, bármilyen platformról is legyen szó [12].

A funkció amely nagyban megkönnyíti a fejlesztést és a tesztelést, az a Hot Reload, amely lehetővé teszi a fejlesztők számára, hogy azonnal láthassák a változtatásokat a kódban, anélkül hogy újra kellene indítani az alkalmazást [12]. Ez a Flutter funkció sokáig egyedüli volt a mobilfejlesztésben, az Android és az iOS csak később vezetett be hasonló funkciókat.

A Flutterben való alkotás hatékonyságát tovább növeli az, hogy egy részletes online dokumentációval rendelkezik, és hogy a Pub Package Manager-t (Csomagkezelőt) használja, amely rengeteg előre elkészített csomagot tartalmaz, az egyszerű UI komponensektől kezdve, a komplex háttér folyamatokig. Ehhez csak meg kell keresni a megfelelőt a pub.dev weboldalon és be kell illeszteni az alkalmazásba [12].

Jelen projekt esetében, mivel a webes felületet a Vue.js és a Vuetify keretrendszerekkel készítettem, a Flutter használatakor inkább a mobilos platformokra koncentráltam, tehát az Androidra és az iOS-re. Összességében így kijelenthető, hogy a legnépszerűbb platformok le lettek fedve és bárki, bármilyen eszközről el tudja érni a Bus4U-t.

5.5. Fejlesztői eszközök

A fejlesztés során számos eszközt használtam, amelyek segítettek a munkámat és a hatékonyságomat. A legfontosabb ezek közül a kódszerkesztő, amelynek a Visual Studio Code-ot használtam, mert ez egy könnyű, gyors és hatékony eszköz, amely támogatja a Vue.js és a Flutter keretrendszereket is. A Visual Studio Code egyik legnagyobb előnye, hogy jól testreszabható a rengeteg letölthető kiegészítő által, amelyek segítségével a fejlesztők szinte bármilyen programozási nyelvet és keretrendszert használhatnak benne⁵.

Mivel a projekten nem egyedül dolgoztam, hanem egy kollégámmal, így szükség volt egy eszközre amelyben vezetni tudjuk a feladatokat átlátható módon. Erre a Jira-t használtuk, amely egy online projektmenedzsment eszköz, amelynek segítségével könnyen lehet követni a feladatokat, a fejlesztés állapotát és a határidőket is egy kanban board-on.⁶ Itt felbontottuk az egész projektet kisebb feladatokra, úgynevezett taszkokra, amelyekhez kódneveket rendeltünk hozzá. Ezeknek a formája *BUS-XX*

⁵Visual Studio Code: <https://code.visualstudio.com/>

⁶A projekt Jira oldala: <https://bus4u.atlassian.net/jira>

volt, ahol az XX egy sorszám, amely a feladatok sorrendjét jelöli. A feladatok állapotát a Jira segítségével tudtuk követni, amelyek lehetnek: "To Do", "In Progress" és "Done". A taszkok nevei tartalmazták a feladat rövid leírását, de hozzá lehetett adni bővebb leírást és hozzászólásokat is. Az 5.7 ábrán látható a Jira tábla, amelyen a feladatok vannak elhelyezve.

Projects / Bus4U

Issues

Share Export Go to all issues LIST VIEW DETAIL VIEW

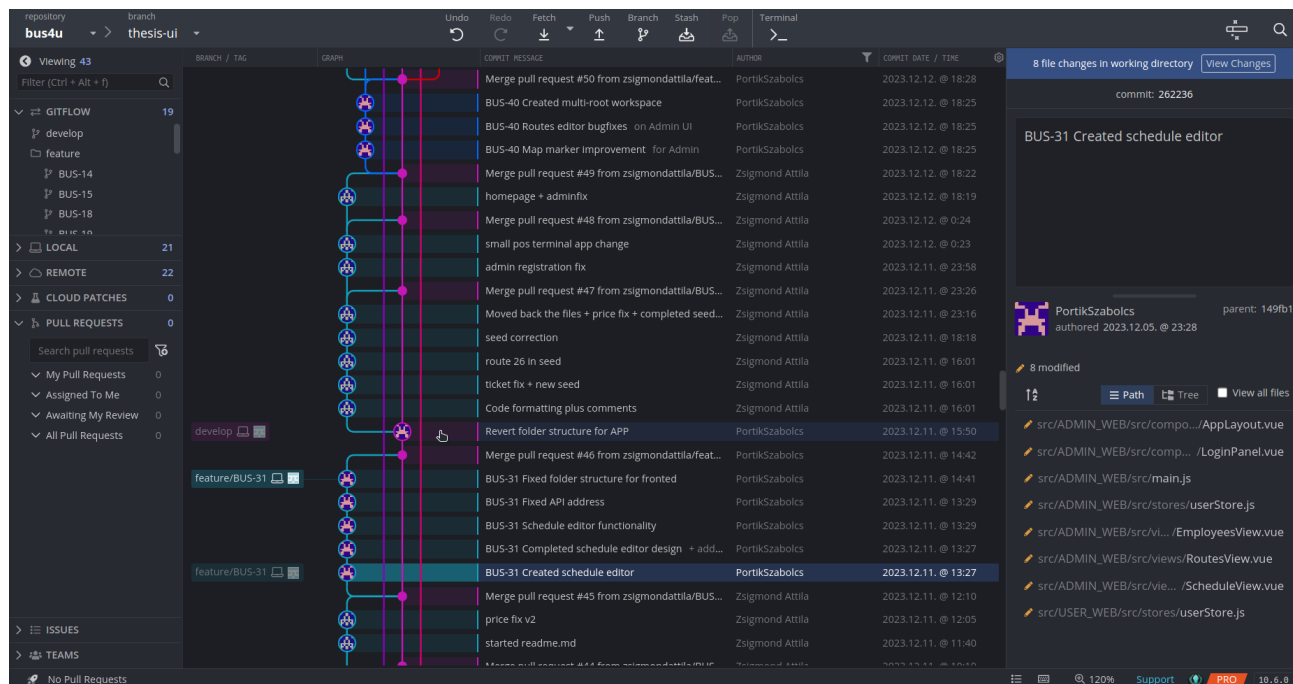
Search issues Project: Bus4U Type Status Assignee: Portik Szabolcs More + Go back to filter Save filter BASIC JQL

Type	Key	Summary	Assignee	Reporter	Priority	Status	Resolution	Created	
☒	BUS-61	Preparations for automatic filling of waybills	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	TO DO	Unresolved	Oct 20, 2024	Oct
☒	BUS-60	Redesign Flutter app	Portik Szabolcs	Portik Szabolcs	Medium	IN PROGRESS	Unresolved	Oct 11, 2024	Oct
☒	BUS-59	Preparations for redeployment on Azure	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	DONE	Done	Oct 3, 2024	Oct
☒	BUS-58	Fix application ticket buying error	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	DONE	Done	Jan 11, 2024	Jan
☒	BUS-56	Final frontend bugfixes	Portik Szabolcs	Portik Szabolcs	Medium	DONE	Done	Jan 3, 2024	Jan
☒	BUS-55	Update methods for admin requests	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	DONE	Done	Jan 3, 2024	Jan
☒	BUS-54	Add token validation to pos_app	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	DONE	Done	Dec 20, 2023	Jan
☒	BUS-51	Request for the number of remaining bus stops	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	TO DO	Unresolved	Dec 19, 2023	Jan
☒	BUS-50	Update methods for elements	Portik Szabolcs	Portik Szabolcs	Medium	DONE	Done	Dec 19, 2023	Jan
☒	BUS-49	Create buses page	Portik Szabolcs	Portik Szabolcs	Medium	DONE	Done	Dec 18, 2023	Dec
☒	BUS-48	Write a base for the documentation	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	DONE	Done	Dec 17, 2023	Dec
☒	BUS-47	Fix POS terminal app bugs	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	DONE	Done	Dec 14, 2023	Dec
☒	BUS-46	Implement actionmailer for the tickets	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	DONE	Done	Dec 13, 2023	Dec
☒	BUS-45	Create employees page	Portik Szabolcs	Portik Szabolcs	Medium	DONE	Done	Dec 13, 2023	Dec
☒	BUS-44	Final touches on the api	Zs. Zsigmond Attila	Zs. Zsigmond Attila	Medium	DONE	Done	Dec 11, 2023	Jan

1-49 of 49

5.7. ábra. Feladatok a Jira-ban

Verziókövetésre a Git-et használtuk, GitHub-al kiegészítve, így biztonságosan tudtuk tárolni a kódot és a változtatásokat is könnyen nyomon tudtuk követni. A GitHub abban is segített, hogy összehangoljuk a frontend és a backend fejlesztését a kollégámmal, valamint hogy több, különböző eszközről is tudjunk dolgozni. A munka során branch-ekre (ágakra) osztottuk a projektet, úgy hogy a fő ág mellett volt egy develop nevű ág és ebből váltak ki az egyes feladatok mellékágai, amelyek mind egy-egy Jira taszknak feleltek meg. A mellékágak nevei tartalmazták a taszk kódját, így össze lehetett hangolni a verziókövetést a kanban táblával. A branchek használata megkönnyítette és strukturáltabbá tette a közös munkát. A Git és a branch-ek egyszerűbb kezelésére a GitKraken nevű eszközt használtam, amely egy grafikus felületet biztosít a Git parancsokhoz, így átláthatóbbá válik a rendszer. A GitKraken kezelőfelülete a projekt ágrajzával az 5.8 ábrán látható.



5.8. ábra. Verziókövetés GitKraken segítségével

5.6. Kitelepítés

A weboldalak ki lettek telepítve Netlify-ra, hogy könnyebben tesztelhetőek és bemutatathatóak legyenek. A Netlify egy olyan szolgáltatás, amely lehetővé teszi a statikus weboldalak gyors és egyszerű kiszolgálását a GitHub repository-ból automatikusan, így a változtatások a *git push* parancs után azonnal megjelennek a weben. A felhasználói weboldal a <https://bus4u.online>, az adminisztrátori weboldal pedig a <https://admin.bus4u.online> címen érhető el.

6. fejezet

A felhasználói felület tervezése

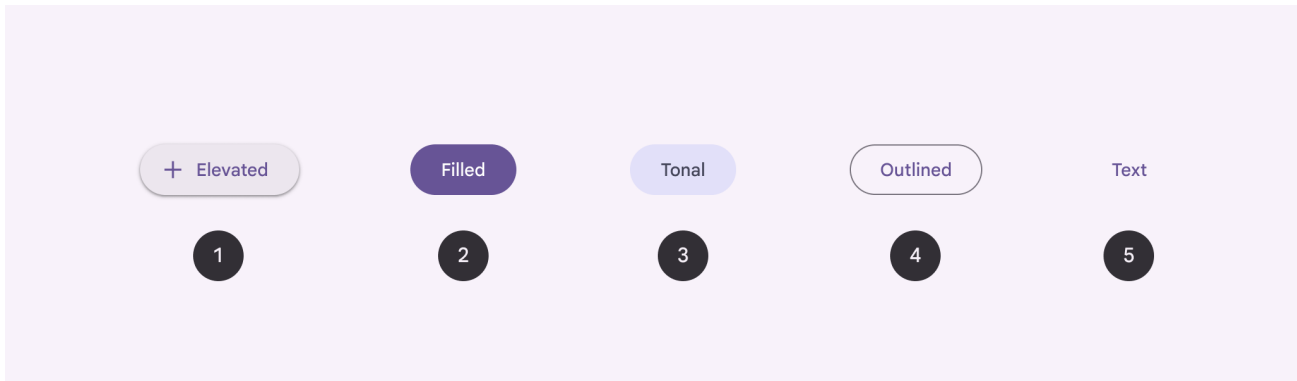
Az egyik legfontosabb része a projektnek a UI, mert ezt a részét látják a felhasználók és ezzel fognak interakcióba lépni. Emiatt kiemelkedően fontos, hogy az jól meg legyen tervezve. Szerencsére ennek a feladatnak a megkönnyítésére léteznek jól bevált módszerek, amelyek nagyban felgyorsítják és megkönnyítik a munkát. Az én választásom a Figma tervezőprogramra és a Material Design 3 irányelveire esett, amelyeket a következőkben fogok bemutatni.

6.1. Material Design

Mivel nehéz egy olyan felhasználói felületi stílust kitalálni, ami tökéletesen működik, egységes és jól is néz ki, ezért a Google Material Design irányelveit követve terveztem meg a felületeket. A Material Design egy olyan nyílt forráskódú dizájn nyelv, amelyet követve egységes, jól kinéző és könnyen használható felületeket lehet készíteni. Ez tartalmazza mindazokat a felületi elemeket, amelyekre szükség lehet egy modern weboldal vagy mobilalkalmazás elkészítésekor, mint például a gombok, ikonok, szövegek, színek, betűtípusok, stb. Mindenről precíz leírás van a hivatalos dokumentációban, így könnyű követni az irányelveket és a felhasználói felület egységességét is biztosítani. Minden komponenshez elérhető egy előre elkészített Figma elem, így ezeket csak be kell húzni a tervezőfelületre, nem kell kézzel létrehozni. Emellett az elemekhez tartozik implementáció is, amely elérhető Flutterben, Androidban és weben is, így a fejlesztés során nem kell lekódolni őket, csak be kell importálni a

projektbe. Ezáltal a fejlesztés gyorsabb lesz és a felhasználói felület egységessége is garantált ¹.

Én a legújabb, 3-as verzióját használom a Material Design-nak, amely a legmodernebb és a legnagyobb támogatással rendelkezik. Ez egy jól bevált UI-t eredményez, amelyet már sok alkalmazás használ, köztük a Google összes alkalmazása is, így már szinte mindenki találkozhatott vele. Egy példa az Material Design 3-as gomb stílusokra a 6.1 ábrán látható. Ilyen és ehhez hasonló komponensekből épül fel az általam tervezett UI is.



6.1. ábra. A Material Design 3 gomb stílusai ²

6.2. Figma terv

A UI terv vizuális elkészítésére a Figma segédprogram a legmegfelelőbb választás rengeteg szempontból. A Figma egy online és offline egyaránt használható felülettervező program, amely lehetővé teszi több felhasználó együttműködését is, például a dizájnerek és a fejlesztők közös munkáját. A Figmában egyszerűen, fogd és vidd módszerrel lehet hozzáadni elemeket a munkavászonhoz, de aki pontosabban szeretne tervezni, annak az oldalsávban van lehetősége pixel pontossággal megadni az elemek tulajdonságait is. Mint említettem, a Figmában elérhetőek előre elkészített Material Design 3-as felületi elemek is, amelyek használata sokszorososan lerövidíti a tervezéshez szükséges időt. A felületi elemekhez funkcionalitást is lehet rendelni, például ha egy gombra kattint a felhasználó, akkor egy másik oldalra navigáljon. A vizuális tervezés mellett lehetőség van a UI terv programozására is, de

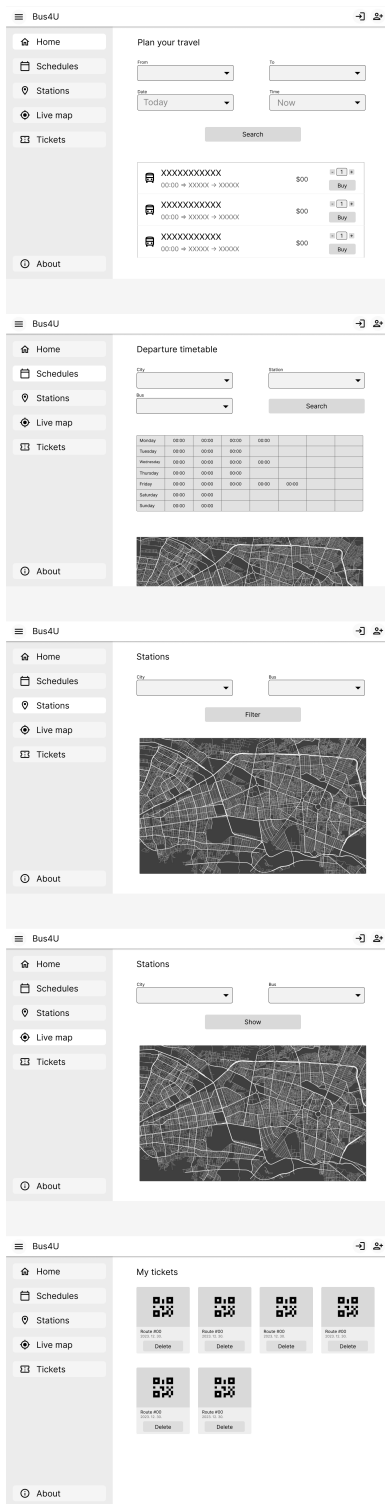
¹Material Design 3: <https://m3.material.io/>

²Forrás: <https://m3.material.io/components/buttons>

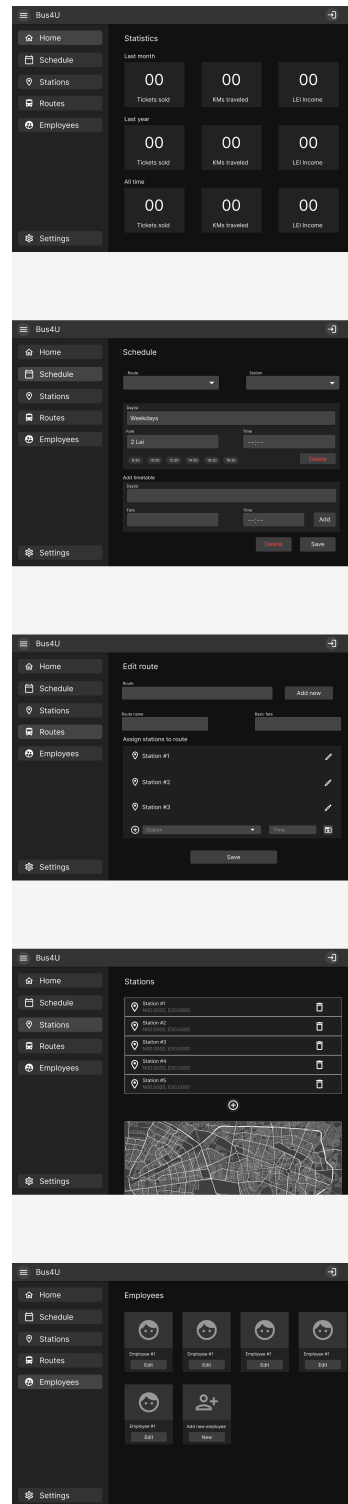
akár a fejlesztéshez szükséges kódot is kigenerálhatjuk az elkészített tervekből. A Figma egy nagyon hasznos eszköz, amely segítségével gyorsan és egyszerűen lehet elkészíteni egy felhasználói felületet, amelyet a fejlesztők is tudnak használni a fejlesztés során ³.

Jelen esetben én vagyok a tervező és a fejlesztő is egyben, így az én feladatom a UI terv elkészítése, de a Figma-t használva, tervezői szaktudás nélkül is sikerült megoldani a feladatot. Az itt létrejött wireframe-eket (drótvázakat) követve már gyorsabb volt a fejlesztés. Munka közben jöttek további ötletek is, így a végső termék nem teljesen egyezik a tervvel, de iránymutatásnak megfelelt. A Material Design irányelvei alapján Figma-ban elkészített felhasználói weboldal terve a 6.2 ábrán, az adminfelület terve a 6.3 ábrán, míg a mobilalkalmazás terve a 6.4 ábrán látható.

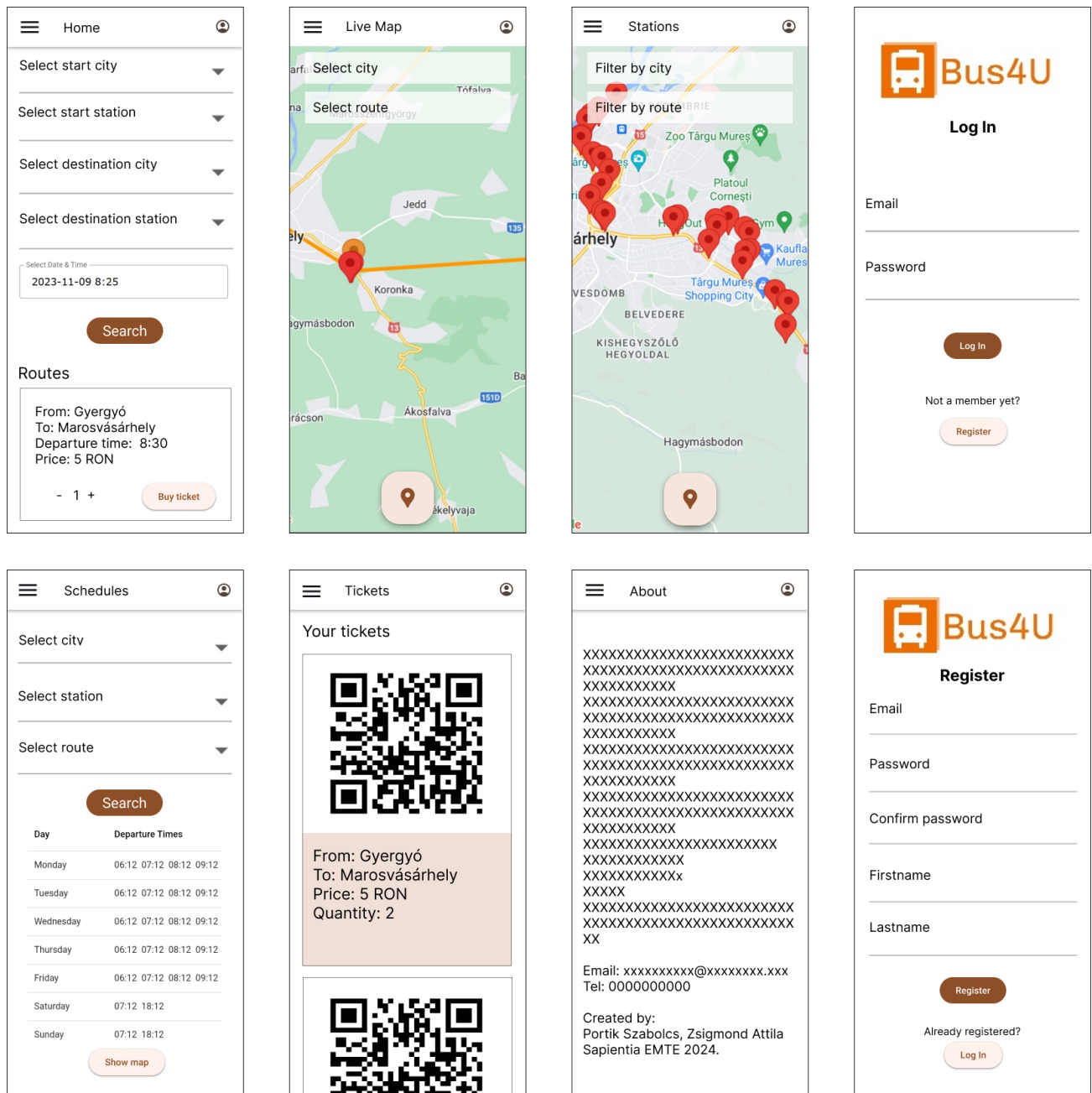
³Figma: <https://www.figma.com>



6.2. ábra. A felhasználói weboldal terve



6.3. ábra. Az admin weboldal terve



6.4. ábra. A mobilalkalmazás terve

7. fejezet

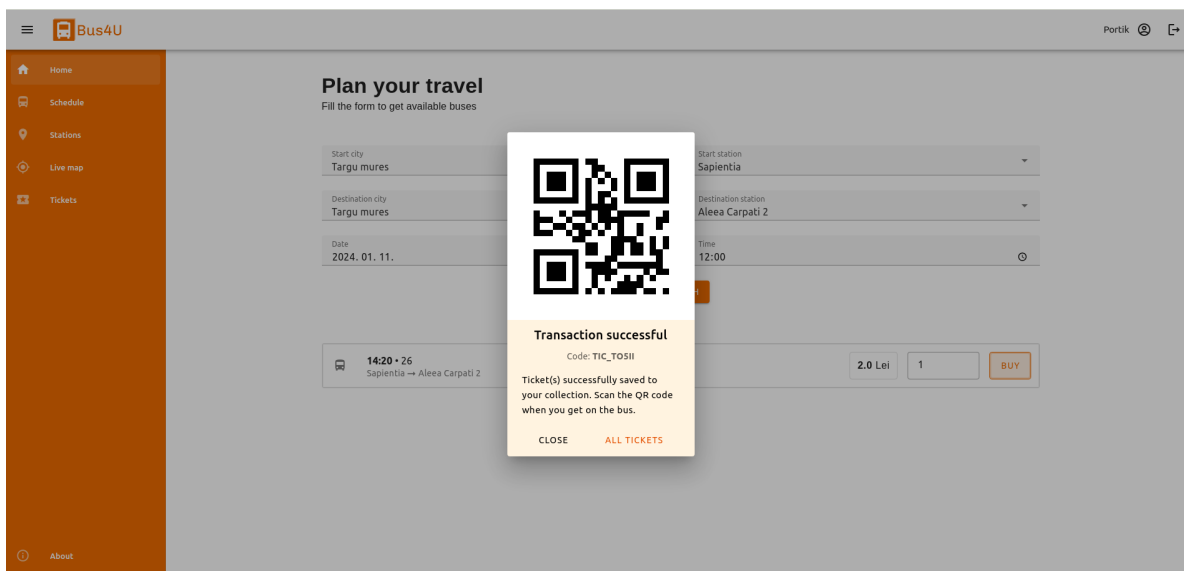
Gyakorlati megvalósítás

7.1. Felhasználói weboldal és alkalmazás

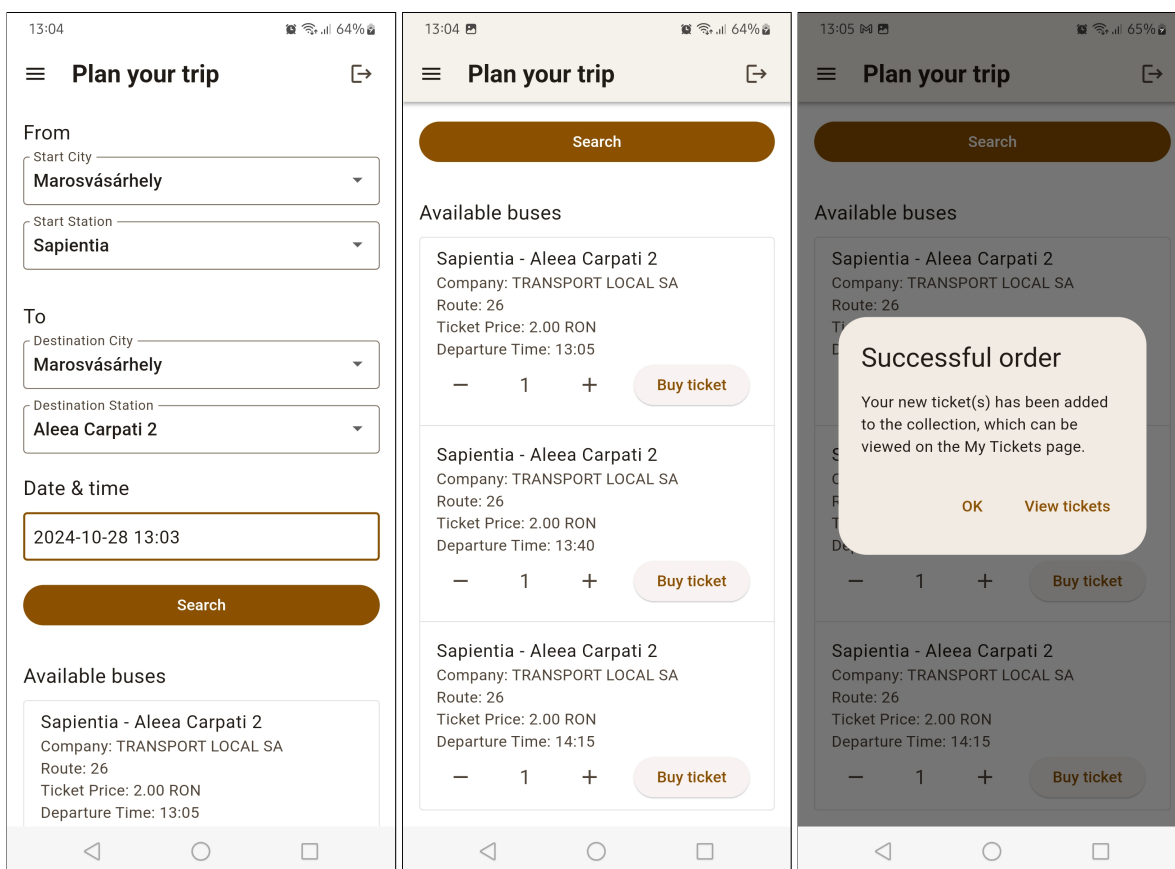
A felhasználói felület működése megegyezik a weboldalon és a mobilalkalmazásban is, azonban a megjelenésük kicsit eltér egymástól, mivel a weboldal a böngészőben fut, míg az alkalmazás a mobil eszközön. Ettől függetlenül a weboldal is tökéletesen használható az okostelefonokon, mivel responszívra terveztem és alkalmazkodik bármilyen kijelzőmérethez, így aki nem szeretne alkalmazást telepíteni, az is könnyen hozzáférhet a Bus4U-hoz böngészőből. A felületek létrehozásakor törekedtem a hasonlóságra, így a navigációs menü, a UI komponensek és a színpaletta is megegyezik, így a felhasználó könnyen átláthatja és használhatja mindkét platformot.

7.1.1. Főoldal

A weboldalra rákeresve a kezdőoldal fogad, ahol lehetőség van egy utazás megtervezésére. Ehhez ki kell választani a kiindulási helyet és megállót, majd a cél helyét és a megállóját, végül pedig az utazás dátumát és időpontját. Ekkor a rendszer lekéri a szerverről a megadott megállók közötti, az adott napon és a megadott időpont után közlekedő járatokat, valamint azok jegyárát. Minden egyes megjelenő járatnál ki lehet választani a jegyek számát, amely alapértelmezetten egy jegyre van beállítva. Ezt követően a felhasználó, a vásárlás gombra kattintva megszerezheti a jegyet vagy jegyeit az adott buszjáratra, amennyiben be van jelentkezve, ellenkező esetben a rendszer átirányítja a bejelentkezéshez.

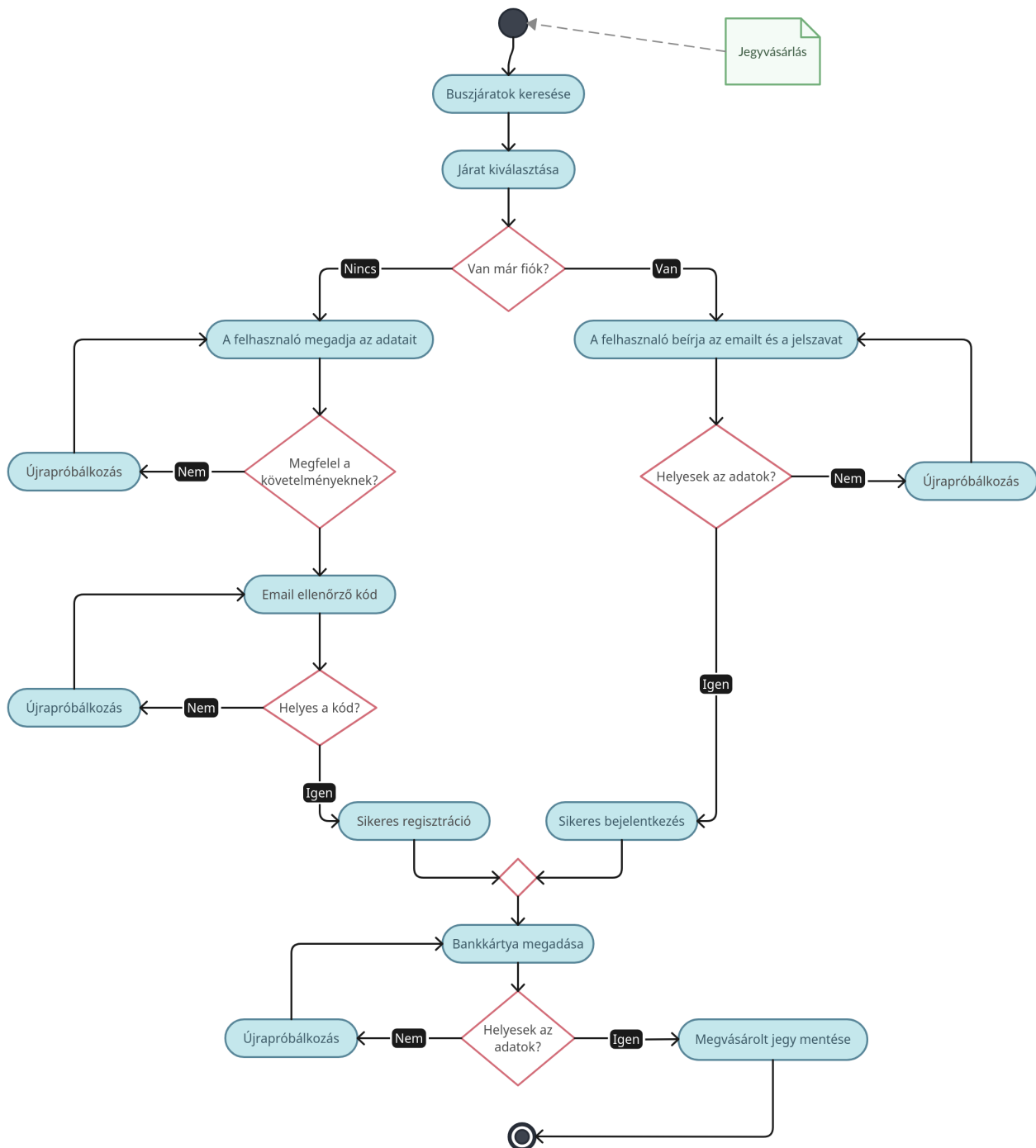


7.1. ábra. Jegyvásárlás a weboldalon



7.2. ábra. Jegyvásárlás a mobilalkalmazásban

A jegyvásárlás folyamata a 7.1 és 7.2 ábrákon látható. A könnyebb megértés végett, a jegyvásárlási és bejelentkezési folyamat szemléltetésére elkészítettem a 7.3 ábrán lévő aktivitás diagramot:



7.3. ábra. A jegyvásárlás folyamata

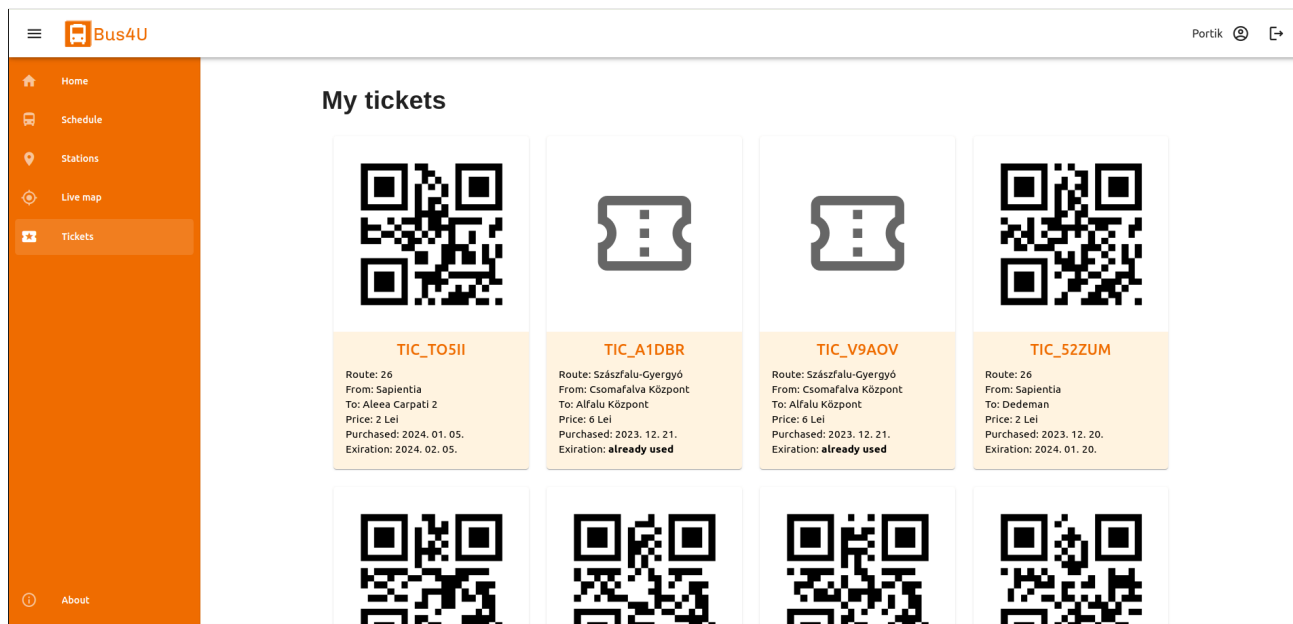
Ahogy az ábrán is látható, miután a felhasználó megtalálta a megfelelő buszjáratot és rákattintott a jegy megvásárlására, a rendszer átirányítja őt a bejelentkezéshez, ahol, ha van már fiókja, bejelentkezhet a meglévő adataival, amennyiben nincs, úgy átléphet a regisztrációs oldalra, ahol meg kell adnia a nevét, email címét, és egy új jelszót. A rendszer minden adatot ellenőriz és ha valamelyik hibás (pl. érvénytelen email cím vagy túl rövid jelszó), akkor egy ennek megfelelő hibaüzenetet ír ki. Ez addig ismétlődik, amíg minden adat megfelelő lesz, aztán a szerver automatikusan küld egy 4 számjegyű álló ellenőrző kódot a megadott e-mail címre, amelyet a felhasználó be kell írjon az alkalmazásban megjelenő szövegmezőbe. Ha ez a kód helytelen, akkor a folyamat kezdődik előlről, a felhasználónak új adatokat kell megadnia. Ha a regisztráció sikeres és az ellenőrző kód helyes, akkor a főoldalra való visszatérés és a jegyvásárlásra kattintás után, a rendszer átirányítja a felhasználót az online fizetési oldalra. A rendszer ott is addig kéri az újrapróbálkozást, amíg minden fizetési adat helyesen lesz megadva. Sikeres kifizetés után a backend generál egy egyedi kódot a megvásárolt jegyhez és megjelenik a kijelzőn a sikeres vásárlás visszaigazolása. Az itt megvásárolt jegye(ke)t a felhasználó a Jegyek nevű oldalon találja meg szöveges és QR-kódos formátumokban.

7.1.2. Jegyek

A megvásárolt jegyeket a rendszer elmenti a felhasználó gyűjteményébe, amely a Jegyek oldalon található, így ezek bármikor visszakereshetőek és felhasználhatóak lesznek, kivéve ha lejár az egy hónapos érvényesség. A jegyen megjelenített adatok között van a járat neve, a kiinduló- és célállomás, az ár, valamint a vásárlás és az érvényesség dátumai. A már elhasznált jegyek megvannak jelölve és el van rejtve a rajtuk lévő QR-kód így azokat nem lehet többször felhasználni. A jegyek a létrehozási idejük szerinti csökkenő sorrendben vannak listázva, így a legfrissebb jegyek mindig az elején vannak.

A itt megjelenő jegyeket úgy lehet felhasználni, hogy a buszra való felszálláskor a sofőr mellett elhelyezett POS terminálra kell mutatni az aktuális jegy QR-kódját, az pedig beolvassa és elküldi a szervernek ellenőrzésre. Ha a kód érvényes és minden rendben van, akkor a felhasználó felszállhat a buszra, a jegy pedig érvénytelenítődik.

A webfelületen listázott jegyek a 7.4 ábrán láthatóak.



7.4. ábra. A felhasználó megvásárolt jegyei

7.1.3. Menetrend

A következő oldal a Menetrend, amely hasonlóan az egyes buszmegállókban kihelyezett táblázatokhoz, megjeleníti a kiválasztott buszjárat indulási időpontjait, az adott megállóból, napokra lebontva. A mi megoldásunk viszont tartalmaz emellett egy interaktív térképet is, ahol a felhasználó meg tudja tekinteni a kiválasztott járat útvonalát, az érintett megállókat és az útvonalon közlekedő buszok élő helyzeteit is, a saját pozíciója mellett. Egy példa a menetrendre és a térképen megjelenő útvonalra a 7.5 ábrán látható.

Bus4U

Portik

Schedule

Select a station and a bus to view the departure timetable and the route marked on the map

City
Marosvásárhely

Station
Dedeman

Bus
Gyergyó-Marosvásárhely

SEARCH

Departure times for station: Dedeman and bus: Gyergyó-Marosvásárhely		
Monday	07:45	19:45
Tuesday	07:45	19:45
Wednesday	07:45	19:45
Thursday	07:45	19:45
Friday	07:45	19:45

☒ Current location
☒ Live bus location
☒ Stations

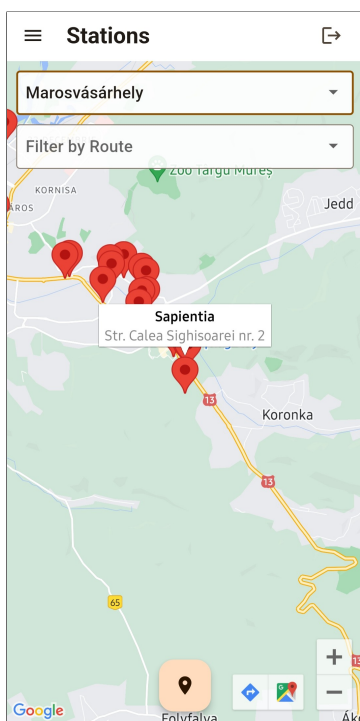
7.5. ábra. Listázott menetrend és az útvonal térképe

7.1.4. Megállók és élő térkép

Ez a két oldal az előbb említett térképes funkciókat mutatják meg külön-külön, egyszerűbben. A Megállók oldal megjeleníti a térképen az összes megállót, és ezek között szűrést is biztosít város vagy buszjárat szerint. Az egyes megállókra kattintva a térképen megjelenik egy kis buborék a kiválasztott megálló adataival, mint a név és a cím, így könnyebb megtalálni a keresett megállót. A térképen megjelenő megállókról példa a 7.6 ábrán látható.

Az élő térkép oldalon pedig egy-egy útvonalon közlekedő buszok élő pozícióját lehet követni, miután kiválasztottuk a várost és az útvonalat. Ezeket az adatokat a buszokon lévő POS terminálok szolgáltatják, így biztosak lehetünk abban, hogy a buszok helyzetei valós időben jelennek meg. Ez a funkció abban segít, hogy ne maradjunk le, ha egy busz siet, illetve ne várakozzunk feleslegesen a megállóban, ha egy busz késik, így időt lehet vele spórolni.

Mindkét oldalon megjelenik a felhasználó jelenlegi pozíciója egy kék gombostű formájában, ha engedélyezte az ehhez való hozzáférést a telefonján, ezáltal könnyebben meg lehet találni a legközelebbi megállót vagy buszt.



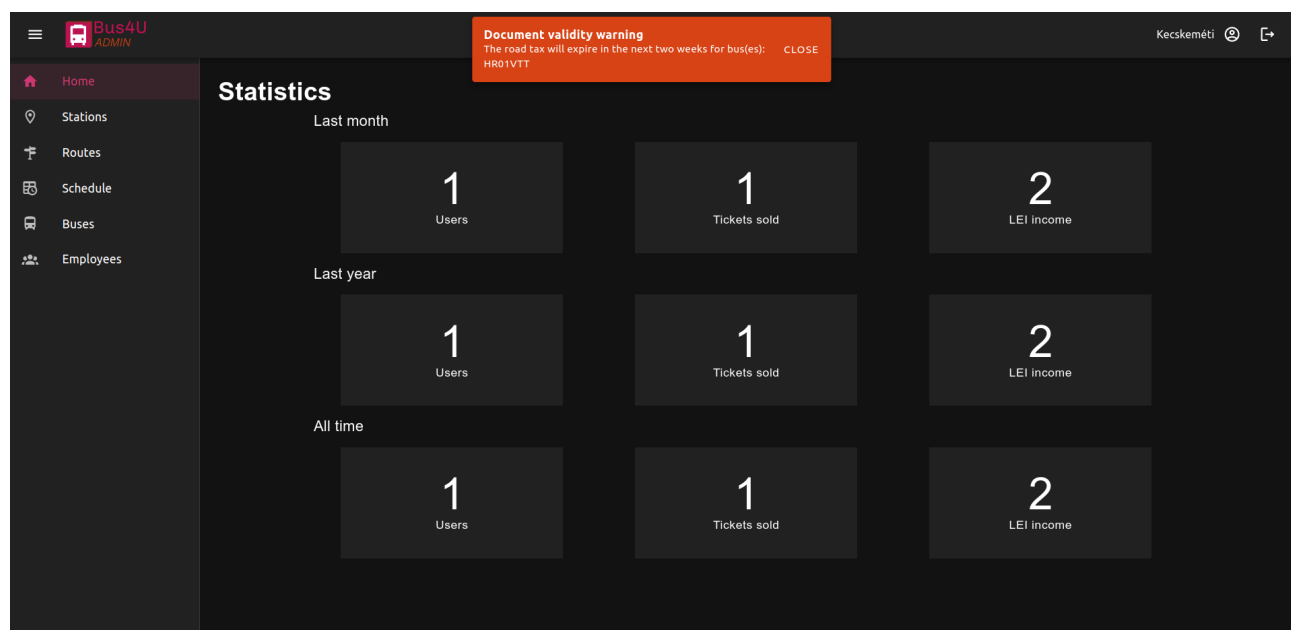
7.6. ábra. Megállók

7.2. Adminisztrátori weboldal

Az adminisztrátori felület csak bejelentkezéssel elérhető, ehhez pedig egy admin típusú felhasználói fiók szükséges, amely a kezelendő céghez van hozzárendelve. A cégek adatai el vannak különítve, így minden adminisztrátor csak a saját cégéhez tartozó adatokat láthatja és módosíthatja. Egy cégnek végtelen számú admin fiókja lehet, viszont ezek között lehetnek különbségek, amelyet a szerepkörök határoznak meg. Egyes adminok csak bizonyos funkciókat érhetnek el, míg mások mindenhez hozzáférhetnek. A sofőr szerepkörben lévő felhasználó például csak a buszok adatait láthatja és kezelheti, míg a menedzser szerepkörű adminisztrátor minden egyéb funkcióhoz is hozzáfér.

7.2.1. Kezdőoldal

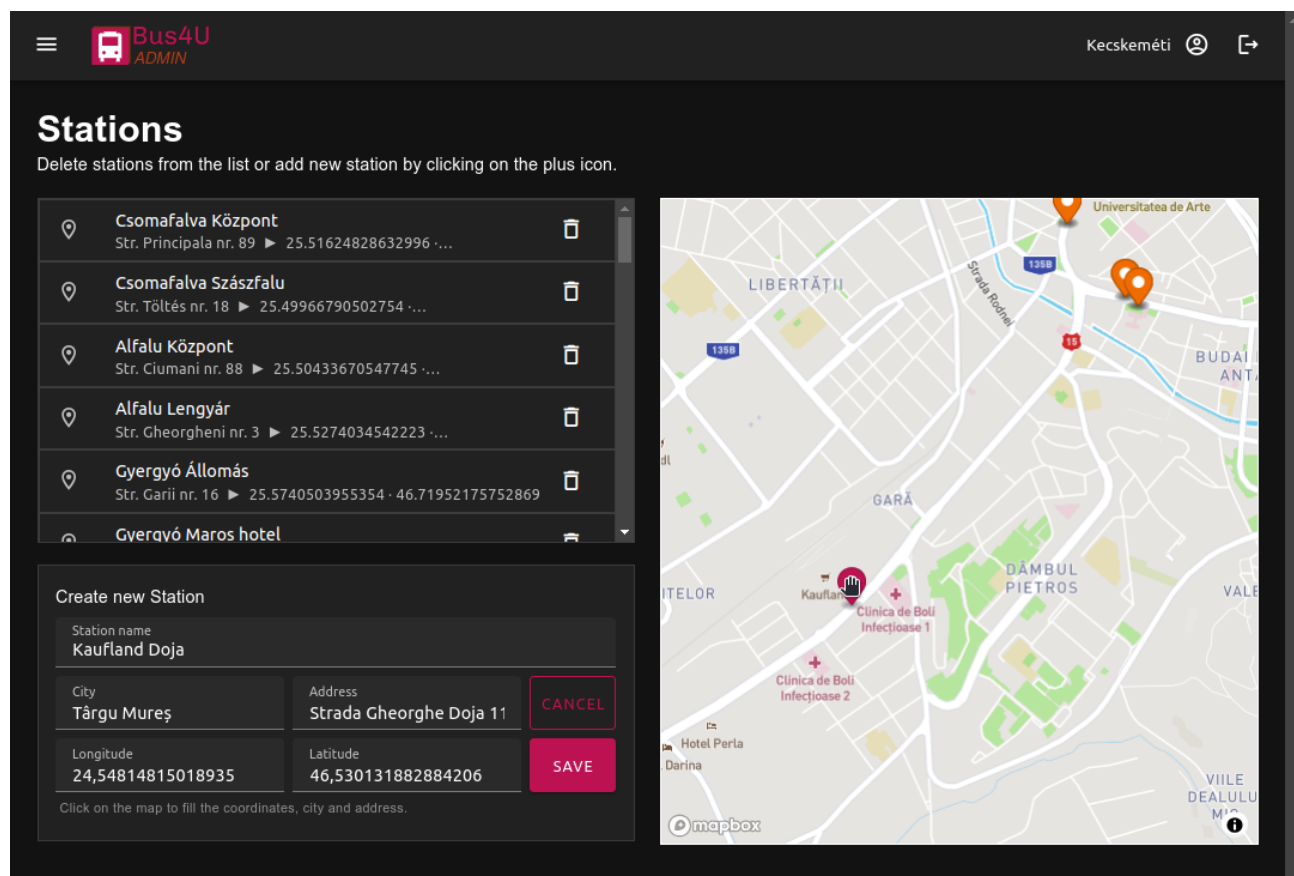
Sikeres bejelentkezés után a kezdőoldal jelenik meg, amelyen látható egy kis statisztika a regisztrált felhasználók számával, a megvásárolt jegyek számával és a bevétellel, mindezek havi, évi és mindenkor bontásban. Ezek segíthetnek a cégvezetőknek a tanulságok levonásában és a fontos döntések meghozatalában is. Ezen kívül itt jelennek meg az esetleges figyelmeztetések és értesítések is, mint például a buszok lejárt útdíja vagy a közelgő műszaki vizsgálat. A kezdőoldal a 7.7 ábrán látható.



7.7. ábra. Kezdőoldal statisztikával és értesítéssel

7.2.2. Megállók

A Megállók oldalon egy lista fogad a meglévő megállókkal és azok adataival, mint a név, a hely és a földrajzi koordináták. Egy megállóra rákattintva ennek pontos helyzete megjelenik a lista mellett található interaktív térképen. A lista elemein található egy törlés gomb is amellyel megszüntethetjük az adott megállót. A lista alatt létre lehet hozni új megállókat is, ehhez csak meg kell adni a megálló nevét, a várost, a címet és a koordinátákat. A rendszer képes automatikusan is kitölteni a néven kívül az összes adatot, ha a felhasználó a megálló helyére kattint a térképen, ahogy ez 7.8 ábrán is látszik.

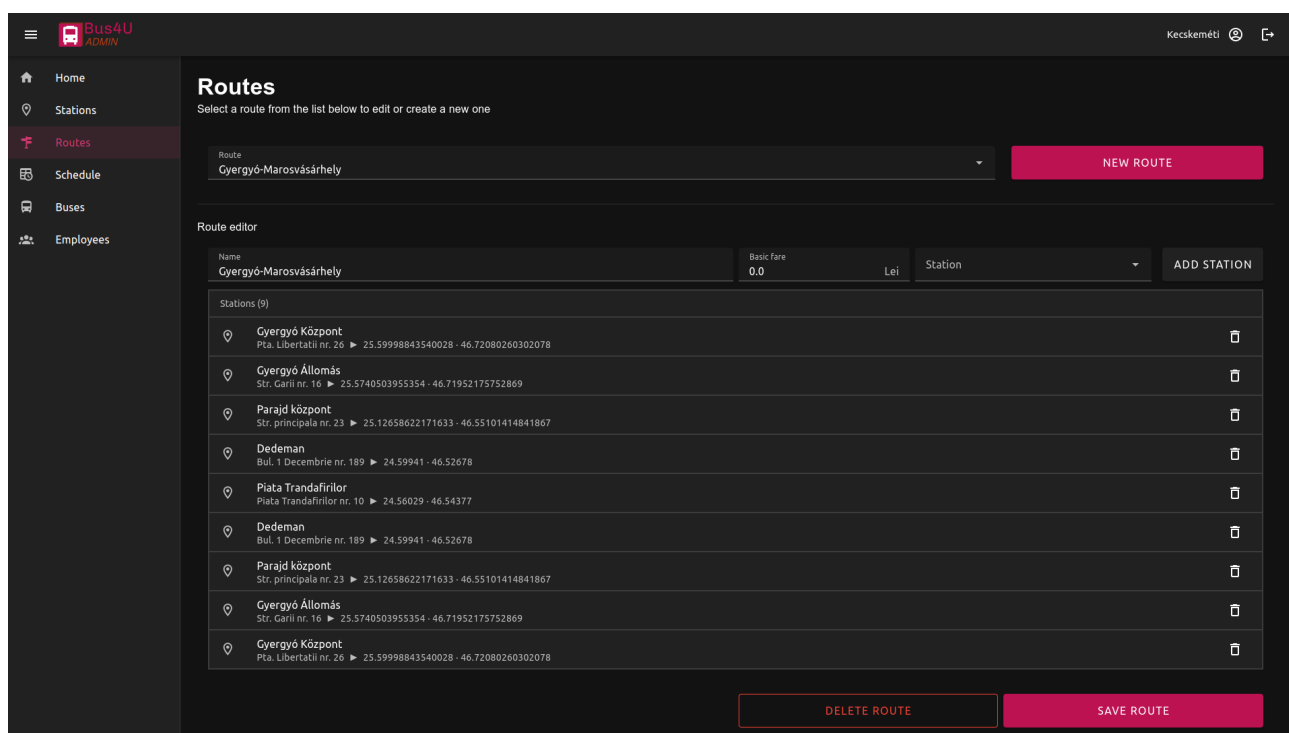


7.8. ábra. Adminisztrátori megállók oldal

7.2.3. Útvonalak

Az Útvonalak oldalon lehet szerkeszteni a buszjáratok útvonalait vagy újakat összerakni a létező megállókból és itt lehet megszabni a járatok alap jegyárát is. Először ki kell választani egy lenyíló listából

a szerkeszteni kívánt útvonalat vagy a mellette lévő gombbal újat lehet hozzáadni. Ezután megjelenik az útvonal-szerkesztő, ahol meg lehet adni a nevet, az alap jegyárat és egy lenyíló listából egyesével ki lehet választani az előző oldalon létrehozott megállókat. A járatok alap jegyára kezdésből 0.0 értékű, mert ez csak olyan esetekben használatos, ahol nem a megállók közötti távolság alapján számolják a jegyárat, hanem fix áron. Ilyen esetben ez az ár minden jegyre érvényes, indulási és érkezési helytől függetlenül. Alább megjelenik a hozzáadott megállók listája, az oldal legalján pedig el lehet menteni a kiválasztott útvonalat vagy akár ki is lehet törölni, ha már nem használják. Az útvonal-szerkesztő és egy betöltött útvonal a mellékelt 7.9 ábrán látható.



7.9. ábra. Útvonalak létrehozása, módosítása és törlése

7.2.4. Menetrend

A következő oldal a Menetrend, ahol a létrehozott útvonalak megállóihoz lehet órarendeket hozzárendelni. Itt az útvonal és megálló kiválasztása után meg kell adni az órarendre érvényes napokat, a következő megállóig megszabott jegyárat és az indulási időpontokat. Egy megállóhoz hozzá lehet adni több ilyen órarendet is, így el lehet különíteni a hétvégét a hétköznapiaktól, de akár minden napra

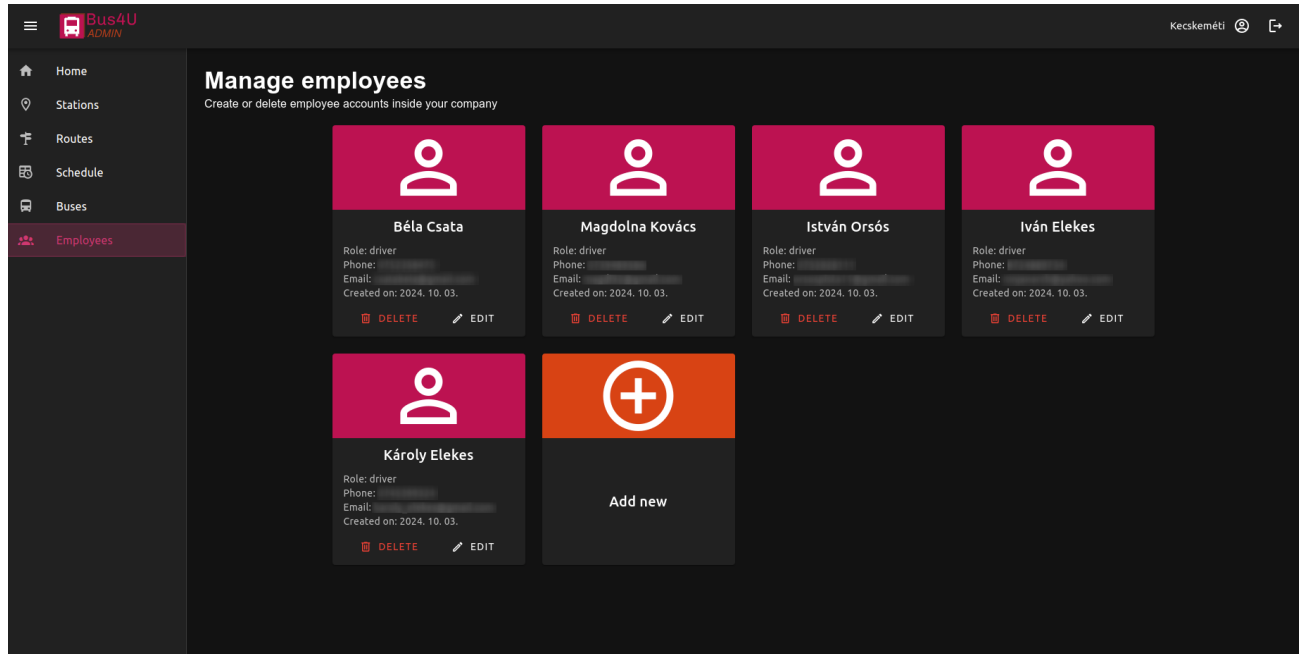
lehet más-más órarendet is megadni, ha különböznek az indulási időpontok vagy akár a jegyárak. Az órarendeket egyesével lehet szerkeszteni és törölni is, csak az a fontos, hogy a módosításokat a felhasználó az oldal alján található gombbal mentse el, különben nem lesznek érvényesek. A 7.10 ábrán látható egy példa a menetrend szerkesztésére.

7.10. ábra. Menetrend szerkesztése

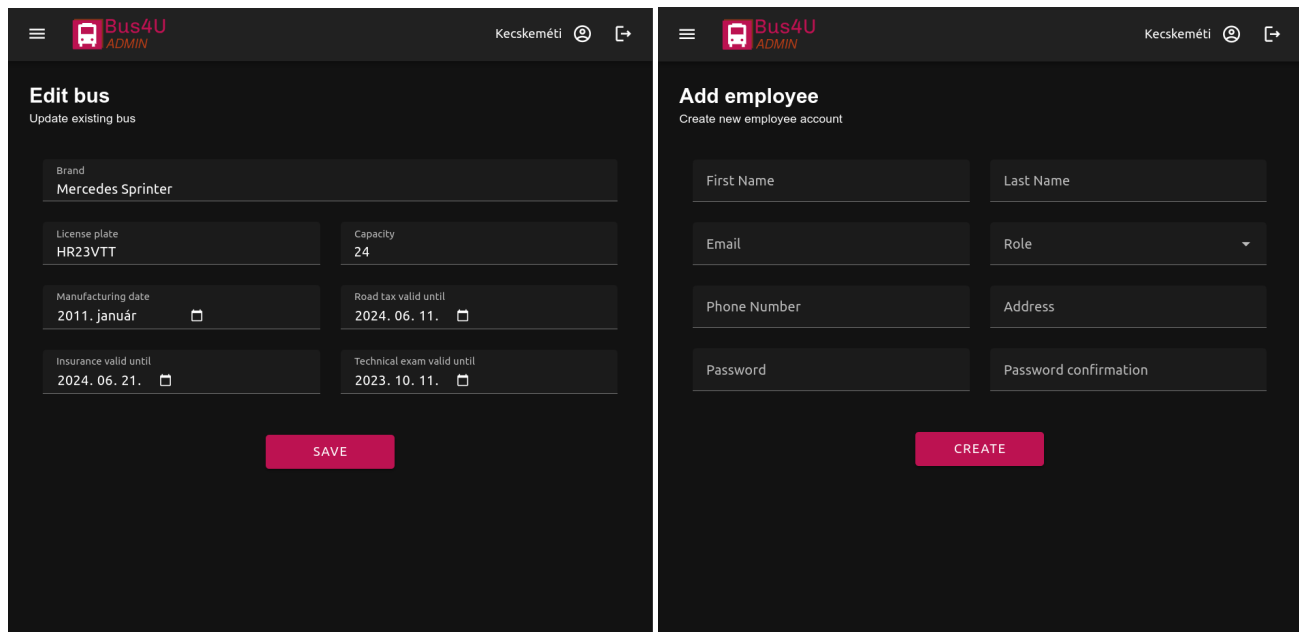
7.2.5. Buszok és alkalmazottak

Az utolsó két oldalon a buszokat és az alkalmazottakat lehet kezelni. Ez a két oldal hasonlóan néz ki és hasonlóan működik. Mindkét esetben látható egy lista a meglévő buszokkal vagy alkalmazottakkal, ahol minden elemet egy ú.n. kártya komponens reprezentál. A kártyák tartalmazzák az elemek összes tulajdonságát, és két gombot, amelyekkel lehet módosítani vagy törölni őket. Az utolsó kártya segítségével újabb példányt is lehet létrehozni az adott kategóriában. A buszok létrehozásánál meg kell adni a márkát, a rendszámot, a férőhelyek számát, a gyártási évet, valamint az útdó, a biztosítás és a műszaki engedély lejárat dátumát. Az utóbbiakról később emlékeztető fog megjelenni a weboldalon, amikor közeledik azok lejárat ideje. Új alkalmazott hozzáadásánál meg kell adni a nevet, e-mail címet, beosztást (ez adja majd meg a jogait a weboldalon), telefonszámot, lakcímet (ez opcionális), és egy új jelszavat. Az alkalmazottak listája és a rajta szereplő kártyák a 7.11 ábrán láthatóak, míg a busz

szerkesztésére és az alkalmazott létrehozására példa a 7.12 ábrán.



7.11. ábra. Az alkalmazottak listája



7.12. ábra. Busz szerkesztése és alkalmazott létrehozása

8. fejezet

Tesztelés

A szoftverrendszerek tesztelése egy nagyon fontos lépés a megfelelő minőség és megbízhatóság biztosításának szempontjából. Célja minimálisra csökkenteni a hibák számát, és biztosítani, hogy a szoftver a tervezett módon működjön. Minél több használati esetet és forgatókönyvet tesztelünk le, annál hibátűrőbbnek tekinthető az adott rendszer. Emellett fontos, hogy az esetleges hibákra minél hamarabb fény derüljön, mert minél később kerülnek kijavításra, annál többre fognak kerülni. Szerencsére napjainkban már számos tesztelési keretrendszer áll rendelkezésre, amelyek segítségével egyszerűen és hatékonyan tesztelhetjük a szoftverünket, sőt már automatizálni is tudjuk ezt a folyamatot.

A projekthez tartozó felhasználói és admin weboldalak tesztelésére a Cypress¹ keretrendszert használtam, mivel nagyon jól integrálható a Vue keretrendszerbe. Ennek segítségével 20 különböző tesztet készítettem, amelyek összesen 65 tesztesetet fednek le. Egy teszt egy teljes funkcionalitás tesztelését jelenti, mint például a bejelentkezési folyamat, míg a teszt eset ennek egy specifikus lefutása, mint például a bejelentkezési kísérlet helytelen adatokkal. Mivel egy adott funkcionalitást általában többféleképpen lehet használni, ebből következik, hogy egy teszt több különböző tesztesetet is tartalmazhat. A UI teszteknek két típusát különböztethetjük meg: komponens és end-to-end (végponttól-végpontig). A komponens tesztek során az egyes komponenseket teszteljük külön egységekként, míg az end-to-end tesztek során a teljes alkalmazást teszteljük, azaz a komponensekből felépített felhasználói felületet. Most pedig következzen a Cypress bemutatása és a megvalósított tesztek részletes leírása.

¹Cypress: <https://www.cypress.io/>

8.1. Cypress

A Cypress egy JavaScript alapú tesztelési keretrendszer, amely a böngészőben fut, és lehetővé teszi a felhasználói felületek tesztelését. Elérhető Windows, macOS és Linux operációs rendszerekre és támogatja a legnépszerűbb böngészőket, de rendelkezik saját beépített böngészővel is. A keretrendszer alkalmas end-to-end tesztek írására és a nagyobb JavaScript keretrendszerek esetében, mint amilyen a Vue is, komponens tesztek is futtathatóak benne. Főbb erősségei a következők:

- **Időutazás:** A Cypress minden egyes lépésről képernyőképet készít és elmenti az adott időpont-hoz tartozó állapotot, így lehetőség van visszalépni az időben a tesztek lefutása után is.
- **Hálózati forgalom vezérlése:** A Cypressben lehetőség van a hálózati forgalom figyelésére és manipulálására is, így akár backend nélkül is futtathatóak a tesztek.
- **Kémfunkciók:** Figyelhetőek és módosíthatóak belső függvények, változók és események is a tesztek során.
- **Egyszerűség és hatékonyság:** A Cypress egyszerűen telepíthető és használható a Node Package Manager (NPM) által, és a tesztek gyorsan futnak, mivel a tesztelés a böngészőben történik.

8.2. Megvalósított tesztek

A weboldalak komponenseinek teszteléséhez 5 komponens tesztet készítettem, amelyek a térképet és a különböző listaelemeket tesztelik, mint például a buszok, a jegyek, vagy az elérhető járatok listájának elemei. Ezek összesen 12 tesztesetet eredményeznek, amelyek a komponensek helyes működését ellenőrzik. Egy példa komponens tesztre, amely egy jegyet megjelenítő felületi elemet ellenőriz, a következő lépéseket tartalmazza:

- A TicketCard komponens renderelése a jegy adataival.
- Ellenőrzés, hogy érvényes jegy esetén a QR kód, míg érvénytelen jegy esetén egy helyettesítő ikon jelenik meg.
- Ellenőrzés, hogy a jegy érvényességi állapota helyesen jelenik-e meg.

Ehhez a komponens teszthez a következő kódot használtam:

```
const ticket = { ticket_uid: "TIC_XXXXX", [...] };
it("renders valid ticket", () => {
  cy.mount(TicketCard, { props: { ticket } });
  cy.get("img").should("exist");
  cy.get(".mdi-ticket-confirmation-outline").should("not.exist");
  cy.get(".v-card-text > :nth-child(6)").should("not.contain", "already used");
});
it("renders invalid ticket", () => {
  ticket.is_valid = false;
  cy.mount(TicketCard, { props: { ticket } });
  cy.get("img").should("not.exist");
  cy.get(".mdi-ticket-confirmation-outline").should("exist");
  cy.get(".v-card-text > :nth-child(6)").should("contain", "already used");
});
```

A másik tesztípus az end-to-end teszt, amely során konkrét használati eseteket teszteltem le, mint például a jegyvásárlás egy kiválasztott buszjáratra. A két weboldalhoz 15 ilyen tesztet készítettem, amelyek összesen 52 tesztesetet tartalmaznak. Ezek ellenőrzik a rendszer összes használati esetének helyes működését, amelyeket korábban megfogalmaztam és amelyek a 4.1 és a 4.2 ábrákon is láthatóak. A felhasználói weboldal end-to-end tesztjeinek listája és eredményei a 8.1 ábrán láthatóak.

Spec		Tests	Passing	Failing	Pending	Skipped
✓ about-page.cy.js	00:01	2	2	-	-	-
✓ home-page.cy.js	00:21	3	3	-	-	-
✓ live-page.cy.js	00:04	2	2	-	-	-
✓ login.cy.js	00:06	4	4	-	-	-
✓ register.cy.js	00:07	3	3	-	-	-
✓ schedule-page.cy.js	00:04	2	2	-	-	-
✓ stations-page.cy.js	00:07	3	3	-	-	-
✓ tickets-page.cy.js	00:04	3	3	-	-	-
✓ All specs passed!	00:58	22	22	-	-	-

8.1. ábra. A felhasználói weboldal tesztjei

Vegyünk egy példát a végponttól-végpontig tartó tesztre, amely az admin felületen történő megálló létrehozásának és törlésének működését ellenőrzi. A teszt során a következő lépések hajtódnak végre:

- Bejelentkezés az admin felületre.
- Megállók oldal felkeresése.
- Kattintás a megálló hozzáadása gombra.
- Várakozás a térkép betöltődésére.
- A térképen egy pont kiválasztása, ahol a megálló létre lesz hozva.
- A megálló nevének megadása és a mentés gomb megnyomása.
- Ellenőrzés, hogy a megálló sikeresen létrejött-e.
- Kattintás a megálló törlése gombra.
- Ellenőrzés, hogy a megálló sikeresen törölve lett-e.

A teszthez tartozó programkód a következőképpen néz ki:

```
it("add new station and remove it", () => {
  cy.intercept("GET", "https://api.mapbox.com/map-sessions/v1*").as("map");
  cy.login();
  cy.visit("/stations");
  cy.get(".d-flex > .v-btn").click();
  cy.wait("@map");
  cy.get(".mapboxgl-canvas").click(320, 500);
  cy.get("input[name='name']").type("test station");
  cy.get("button[type='submit']").click();
  cy.get(".v-snackbar__content").contains("Station created");
  cy.get(".v-list > .v-list-item").contains("test stat").parent().siblings().find("button").click();
  cy.get(".v-snackbar__content").contains("Station deleted");
  cy.get(".v-list > .v-list-item").should("not.contain", "test station");
});
```

8.3. Automatizált tesztelés

A tesztek futtatásának automatizálását a GitHub Actions segítségével valósítottam meg. A GitHub Actions egy olyan funkció a GitHubon, amely lehetővé teszi előre definiált feladatok automatikus végrehajtását a kód változásának alkalmával. A Cypress biztosít egy előre megírt GitHub Actions konfigurációt, amelyet csak be kell állítani a projektünkben, és a tesztek automatikusan lefutnak minden egyes változtatás után. Az én esetemben 2 ilyen feladatra van szükség: az egyik a felhasználói, míg a másik az admin weboldal tesztjeinek futtatására. Ezekhez hozzáadtam még egy extra lépést, amely hiba esetén lementi a tesztek által készített képernyőképeket utólagos elemzés céljából. Mindhárom tesztelési feladat a "Cypress tests" nevű Action része, amelynek konfigurációs kódja a következő:

```
name: Cypress tests
on: push
jobs:
  cypress-run:
    name: Cypress run
    runs-on: ubuntu-24.04
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Cypress test USER
        uses: cypress-io/github-action@v6
        env:
          VITE_MAPBOX_TOKEN: ${ secrets.VITE_MAPBOX_TOKEN }
        with:
          start: npm run dev
          working-directory: USER_WEB
      - name: Cypress test ADMIN
        uses: cypress-io/github-action@v6
        env:
          VITE_MAPBOX_TOKEN: ${ secrets.VITE_MAPBOX_TOKEN }
        with:
          start: npm run dev
          working-directory: ADMIN_WEB
      - name: Upload screenshots
        uses: actions/upload-artifact@v4
        if: failure()
        with:
          name: cypress-screenshots
          path: |
            USER_WEB/cypress/screenshots
            ADMIN_WEB/cypress/screenshots
```

A tesztek lefutása után a GitHub Actions értesítést küld az eredményekről email-ben, és a webfelületen is megjeleníti ezeket táblázatos formában. Itt amennyiben szükséges, vissza lehet nézni a naplót vagy le lehet tölteni a hibákról készült képernyőképeket, amelyek segítségével könnyen meg lehet tudni, hogy mi okozta az esetleges hibákat. A sikeres tesztek lefutása után megjelenő összefoglaló táblázat a 8.2 ábrán látható.

Cypress run summary					
Cypress Results					
Result	Passed 	Failed 	Pending 	Skipped 	Duration 
Passing 	22	0	0	0	74.8s
Cypress Results					
Result	Passed 	Failed 	Pending 	Skipped 	Duration 
Passing 	30	0	0	0	156.345s
Job summary generated at run-time					

8.2. ábra. A GitHub Actions által generált teszteredmények

9. fejezet

Összefoglalás

Befejezésképpen bátran kijelenthetem, hogy a projekt sikeres, mivel a leszögezett terveket sikerült implementálni, sőt fejlesztés közben egyéb hasznos funkcionalitásokat is sikerült hozzáadni a végső eredményhez. Ezáltal a végfelhasználóknak egy-egy funkciódús és jól működő weboldal és applikáció áll a rendelkezésükre, amely nagyban megkönnyíti a tömegközlekedésben való részvételt, bármilyen eszközön is próbálnák elérni ezt. A felhasználók pillanatok alatt tervezhetik meg az utazásaikat, vásárolhatnak jegyet, informálódhatnak megállókról, útvonalakról és órarendekről, sőt a buszok élő pozíciót is nyomon követhetik a térképen. A busztársaságoknak pedig egy egyszerű vezérlőfelület van létrehozva, amely segítségével könnyen és gyorsan tudják kezelni a buszokat, az útvonalakat, a menetrendeket, a jegyárakat és az alkalmazottakat, az üzleti döntéseik megkönnyítésére pedig egy statisztika is a rendelkezésükre áll.

Következtetésképpen a Bus4U egy reális piaci rést tud betölteni, mivel megoldja a tömegközlekedéssel kapcsolatos problémákat és ezáltal vonzóbbá teszi az ilyen eszközök használatát az emberek számára. Ez azért fontos, mert a buszokkal való közlekedés elengedhetetlen a forgalom, és a környezetszennyezés csökkentéséhez, de jelenleg, a rendszer hiányosságai miatt ez egy nem túl népszerű közlekedési eszköz hazánkban.

9.1. További fejlesztési lehetőségek

Felhasználók szempontjából a következő fontos fejlesztés a buszok telítettségének valós idejű követése lenne, mert ezzel elkerülhető lenne a zsúfoltság a buszokon, és így sokat javulna a tömegközlekedés minősége. Be szeretnénk idővel vezetni a bérletek, illetve különböző kedvezmények kezelésének lehetőségét is, mert ez is növelheti a népszerűséget. Hasznos lenne az útvonaltervezés kibővítése gyalogos navigációval, valamint a távolság, idő és forgalom szempontjából legoptimálisabb útvonal felajánlásával, így az alkalmazás el tudná vezetni a felhasználót a számára legmegfelelőbb megállóig. A különböző értesítések kiküldése is hasznos lehet, például ha egy busz késik, vagy ha egy útvonalon változás történik, fontos lenne, hogy időben informálódjon erről a felhasználó.

A kényelmi funkciók bővítése mellett, be lehetne vezetni egy kis játékosítást is, például a buszokon való utazásokért járó pontokat lehetne összegyűjteni, amelyeket később be lehetne váltani kedvezményekre, vagy elérhetővé tenni különböző mérőföldköveket, amelyeket a felhasználók elérhetnek és gyűjthetnek. A pontok mellett a kilométereket is lehetne gyűjteni, amelyek megjelennének a felhasználók profiljaiban, és így versenyezhetnének egymással. Ezek a játékos megoldások egy kis színt vihetnek az unalmas buszozásba, így nagyban növelhetik a felhasználók aktivitását és hűségét.

A busztársaságok szempontjából is rengeteg lehetőség áll rendelkezésre. Mivel a projektet úgy terveztük, hogy több busztársaság is fel tudja használni, így ez jelen pillanatban általános használatra van optimalizálva. Ennek ellenére egy-egy partner cég kérésére személyre is szabhatnánk a rendszert, egyedi funkciókat és kinézetet bevezetve. Az is hasznos lenne például, ha a cég akár a teljes könyvelését le tudná itt bonyolítani: az alkalmazottak munkaórát automatikusan vezetni, az autóbuszok tankolásait összesíteni, illetve buszokon lévő POS terminál GPS moduljának segítségével akár a "Foire de parours" vezetése is automatizálható lenne. A könyveléshez a járművek szervíz költségeinek számláit is kellene gyűjteni az oldalra, amivel további hasznos kimutatásokat tudnánk generálni.

Ezek az ötletek jutottak eddig eszünkbe, de még rengeteg lehetőség van olyan fejlesztésekre, amelyek növelhetik a Bus4U hatékonyságát és népszerűségét mind a felhasználók, mind a busztársaságok számára.

Irodalomjegyzék

- [1] Z. Attila, „Bus4u - server,” bachelor’s thesis, Sapientia Hungarian University of Transylvania, 2025.
- [2] M. Cropper and P. Suri, „Measuring the air pollution benefits of public transport projects,” *Regional Science and Urban Economics*, vol. 107, p. 103976, 2024.
- [3] T. P. L. Le and T. A. Trinh, „Encouraging public transport use to reduce traffic congestion and air pollutant: A case study of ho chi minh city, vietnam,” *Procedia engineering*, vol. 142, pp. 236–243, 2016.
- [4] M. Orošnjak, M. Jocanović, B. Gvozdenac-Urošević, D. Šević, L. Duđak, and V. Karanović, „Bus fleet management—a systematic literature review,” *Promet-Traffic&Transportation*, vol. 32, no. 6, pp. 761–772, 2020.
- [5] P. Polyviou, *Modelling traffic incidents to support dynamic bus fleets management for sustainable transport*. PhD thesis, University of Southampton, 2011.
- [6] O. Elijah, S. L. Keoh, S. K. bin Abdul Rahim, C. K. Seow, Q. Cao, M. A. bin Sarijari, N. F. Ibrahim, and A. Basuki, „Transforming urban mobility with internet of things: public bus fleet tracking using proximity-based bluetooth beacons,” *Frontiers in the Internet of Things*, vol. 2, p. 1255995, 2023.
- [7] N. Hounsell, B. Shrestha, and A. Wong, „Data management and applications in a world-leading bus fleet,” *Transportation Research Part C: Emerging Technologies*, vol. 22, pp. 76–87, 2012.

- [8] V. Rolland and F. Gábor, „Infrastrukturális vagy közösségi érzékelés az okos városokban?,” *Híradástechnika*, vol. 71, no. 1, pp. 47–51, 2016.
- [9] E. Hanchett and B. Listwon, *Vue.js in Action*. Simon and Schuster, 2018.
- [10] P. D. Garaguso, *Vue.js 3 Design Patterns and Best Practices: Develop scalable and robust applications with Vite, Pinia, and Vue Router*. Packt Publishing Ltd, 2023.
- [11] R. Desai, „Google material design—the user interface development notion,” in *IJSRD National Conference on Advances in Computer Science Engineering & Technology*, pp. 125–7, 2017.
- [12] E. Windmill, *Flutter in action*. Simon and Schuster, 2020.